

CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY
FACULTY OF SOFTWARE ENGINEERING



GRADUATION THESIS
BACHELOR OF ENGINEERING IN SOFTWARE ENGINEERING
(HIGH-QUALITY PROGRAM)

DEVELOPING AN ONLINE APPOINTMENT
MANAGEMENT, TELEHEALTH CONSULTING, AND
HEALTHCARE SUPPORT SYSTEM

Student: Duc Hung Nguyen

Student ID: B2203556

Class: 2022 – 2026 (K48)

Advisor: Ph.D. Thanh Khoa Nguyen

Can Tho, August 2026

CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY
FACULTY OF SOFTWARE ENGINEERING



GRADUATION THESIS
BACHELOR OF ENGINEERING IN SOFTWARE ENGINEERING
(HIGH-QUALITY PROGRAM)

DEVELOPING AN ONLINE APPOINTMENT
MANAGEMENT, TELEHEALTH CONSULTING, AND
HEALTHCARE SUPPORT SYSTEM

Student:

Student name: Duc Hung Nguyen

Student ID: B2203556

Class: DI2296F1, K48

Advisor:

Ph. D. Thanh Khoa Nguyen

Can Tho, August 2026

COMMENTS OF THE ADVISOR

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Can Tho, August ... 2026

Ph. D. Thanh Khoa Nguyen

ACKNOWLEDGEMENTS

First and foremost, I would like to express my deepest gratitude to my advisor, **Ph.D. Thanh Khoa Nguyen**, for his invaluable guidance, continuous encouragement, and constructive feedback throughout the course of this graduation project. His profound academic expertise, professionalism, and patient instruction have been instrumental in shaping the structure, methodology, and technical execution of this thesis.

I would also like to convey my sincere thanks to **the respected faculty members of the College of Information and Communications Technology at Can Tho University**. The fundamental knowledge, technical skills, and software engineering methodologies instilled by my instructors over the past four years served as the solid foundation upon which this complex, microservices-based system was realized.

Furthermore, I am highly appreciative of **the thesis defense committee members** for dedicating their valuable time to evaluate, critique, and provide academic recommendations that have substantially improved the quality of this work.

Finally, I owe an immense debt of gratitude to **my family and friends**, whose unwavering moral and emotional support enabled me to focus entirely on my academic pursuits and successfully bring this research project to fruition.

DECLARATION

I hereby declare that this graduation thesis, titled “Developing an Online Appointment Management, Telehealth Consulting, and Healthcare Support System”, is a product of my own individual research and development under the direct supervision of Ph.D. Thanh Khoa Nguyen.

I solemnly commit that all system architectures, database designs, source code, data analysis, and results presented in this document are genuine, verified, and have not been plagiarized from any other individual, organization, or publication. Any third-party APIs, software libraries, and documentation utilized in the creation of the system are clearly referenced and cited in accordance with academic standards.

Can Tho, August ... 2026

Duc Hung Nguyen

TABLE OF CONTENTS

COMMENTS OF THE ADVISOR.....	i
ACKNOWLEDGEMENTS	ii
DECLARATION.....	iii
TABLE OF CONTENTS	iv
LIST OF FIGURES.....	viii
LIST OF TABLES	ix
LIST OF ABBREVIATIONS AND TECHNICAL TERMS	xi
TÓM TẮT	xiv
ABSTRACT	xv
PART I: INTRODUCTION	1
1. Problem statement and rationale	1
2. Existing solutions and prior works.....	1
3. Research objectives	2
4. Research subject and scope	3
4.1. Research subject.....	3
4.2. Research scope	4
5. Research Content and Technologies Used.....	4
6. Contributions of the thesis.....	6
7. Thesis organization	7
PART II: CONTENT	9
CHAPTER 1: PROBLEM DESCRIPTION.....	9
1. Detailed problem description	9
2. Functional requirements	10
2.1. Use case diagrams	10
2.1.1. General use case diagram.....	10
2.1.2. Patient actor use case diagram	13
2.1.3. Doctor actor use case diagram	15
2.1.4. Admin and Timer actor use case diagram.....	17
2.1.5. Nurse actor use case diagram	19
2.2. Use case specifications.....	20

2.2.1. Book appointment	20
2.2.2. Schedule	22
2.2.3. Clear unused schedule	24
2.2.4. Examine.....	25
2.2.5. Join remote consultation.....	27
2.2.6. Create schedule change request	29
2.2.7. Handle schedule change requests.....	31
2.2.8. Pay invoices online.....	33
2.2.9. View invoices	35
2.2.10. Confirm cash payment	37
3. Non-functional requirements.....	39
3.1. Performance Requirements	39
3.2. Safety requirements.....	39
3.3. Security requirements.....	40
3.4. External interface requirements	41
3.4.1. User interface (UI) requirements	41
3.4.2. Hardware interface requirements	41
3.4.3. Software interface requirements	41
3.4.4. Communication interface requirements	41
3.5. Business rules	42
CHAPTER 2: SYSTEM DESIGN AND IMPLEMENTATION.....	43
1. System architecture	43
2. Data design	46
2.1. Data model	46
2.1.1. user_service.....	46
2.1.2. schedule_service.....	47
2.1.3. medical_record_service.....	47
2.1.4. feedback_service	48
2.1.5. notification_service	48
2.1.6. payment_service.....	48
2.2. Data dictionary	49

2.2.1. user_service	49
2.2.2. schedule_service.....	52
2.2.3. medical_record_service.....	56
2.2.4. feedback_service	59
2.2.5. notification_service	59
2.2.6. payment_service.....	60
3. Detailed functional design.....	61
3.1. Book appointment	62
3.2. Schedule	64
3.3. Clear unused schedule.....	67
3.4. Examine.....	68
3.5. Join remote consultation.....	71
3.6. Create schedule change request.....	72
3.7. Handle schedule change request	75
3.8. Pay invoices online.....	77
3.9. View invoices	79
3.10. Confirm cash payment	82
CHAPTER 3: SOFTWARE TESTING AND PERFORMANCE EVALUATION	84
1. Introduction	84
1.1. Testing objectives:.....	84
1.2. Testing scope.....	84
2. Test plan details.....	84
2.1. Tested functions	84
2.2. Approach methods.....	85
2.3. Test pass / fail criteria	85
2.4. Suspension and re-evaluation criteria	86
2.5. Test deliverables.....	86
2.6. Testing environment.....	86
2.7. Test resources	87
2.8. Testing risks and mitigation plans.....	87
3. Test scenarios	87

4. Test cases.....	89
5. Test result evaluation	94
PART III. CONCLUSION.....	96
1. Achieved results	96
1.1. Personal growth and technical development.....	96
1.2. Program practically and real-world application.....	97
2. Limitations and difficulties	97
3. Future works.....	99
PART IV. REFERENCES	101

LIST OF FIGURES

<i>Figure 1. 1. General use case diagram</i>	<i>11</i>
<i>Figure 1. 2. Patient actor use case diagram</i>	<i>13</i>
<i>Figure 1. 3. Doctor actor use case diagram</i>	<i>15</i>
<i>Figure 1. 4. Admin and Timer actor use case diagram</i>	<i>17</i>
<i>Figure 1. 5. Nurse actor use case diagram</i>	<i>19</i>
<i>Figure 2. 1. System architecture diagram</i>	<i>43</i>
<i>Figure 2. 2. user_service's physical data model</i>	<i>46</i>
<i>Figure 2. 3. schedule_service's physical data model</i>	<i>47</i>
<i>Figure 2. 4. medical_record_service's physical data model.....</i>	<i>47</i>
<i>Figure 2. 5. feedback_service's physical data model</i>	<i>48</i>
<i>Figure 2. 6. notification_service's physical data model</i>	<i>48</i>
<i>Figure 2. 7. payment_service's physical data model</i>	<i>48</i>
<i>Figure 2. 8. Book appointment user interface.....</i>	<i>62</i>
<i>Figure 2. 9. Book appointment process.....</i>	<i>64</i>
<i>Figure 2. 10. Schedule process</i>	<i>66</i>
<i>Figure 2. 11. Clear unused schedule process</i>	<i>67</i>
<i>Figure 2. 12. Examine user interface</i>	<i>68</i>
<i>Figure 2. 13. Examine process</i>	<i>70</i>
<i>Figure 2. 14. Join remote consultation user interface</i>	<i>71</i>
<i>Figure 2. 15. Join remote consultation process</i>	<i>72</i>
<i>Figure 2. 16. Create schedule change request user interface.....</i>	<i>73</i>
<i>Figure 2. 17. Create schedule change request process</i>	<i>75</i>
<i>Figure 2. 18. Handle schedule change request user interface.....</i>	<i>75</i>
<i>Figure 2. 19. Handle schedule change request process.....</i>	<i>77</i>
<i>Figure 2. 20. Pay invoices online user interface</i>	<i>78</i>
<i>Figure 2. 21. Pay invoices online process</i>	<i>79</i>
<i>Figure 2. 22. View invoices user interface 1</i>	<i>80</i>
<i>Figure 2. 23. View invoices user interface 2</i>	<i>80</i>
<i>Figure 2. 24. View invoices process.....</i>	<i>82</i>
<i>Figure 2. 25. Confirm cash payment user interface.....</i>	<i>82</i>
<i>Figure 2. 26. Confirm cash payment process</i>	<i>83</i>

LIST OF TABLES

<i>Table 1. 1. Book appointment use case specification</i>	22
<i>Table 1. 2. Schedule use case specification</i>	24
<i>Table 1. 3. Clear unused schedule use case specification</i>	25
<i>Table 1. 4. Examine use case specification</i>	27
<i>Table 1. 5. Join remote consultation use case specification</i>	29
<i>Table 1. 6. Create schedule change request use case specification</i>	31
<i>Table 1. 7. Handle schedule change request use case specification</i>	33
<i>Table 1. 8. Pay invoices online use case specification</i>	35
<i>Table 1. 9. View invoices use case specification</i>	37
<i>Table 1. 10. Confirm cash payment use case specification</i>	39
<i>Table 2. 1. roles table</i>	49
<i>Table 2. 2. degrees table</i>	49
<i>Table 2. 3. specialties table</i>	50
<i>Table 2. 4. users table</i>	51
<i>Table 2. 5. doctor_metadata table</i>	52
<i>Table 2. 6. opening_time table</i>	52
<i>Table 2. 7. global_holidays table</i>	53
<i>Table 2. 8. doctor_weekly_templates table</i>	53
<i>Table 2. 9. time_slots table</i>	54
<i>Table 2. 10. bookings table</i>	54
<i>Table 2. 11. doctor_leaves table</i>	55
<i>Table 2. 12. schedule_change_requests table</i>	56
<i>Table 2. 13. schedule_change_request_details table</i>	56
<i>Table 2. 14. diseases table</i>	57
<i>Table 2. 15. medical_records table</i>	57
<i>Table 2. 16. medicines table</i>	58
<i>Table 2. 17. prescription_details table</i>	58
<i>Table 2. 18. user_feedbacks table</i>	59
<i>Table 2. 19. notifications table</i>	59
<i>Table 2. 20. invoices table</i>	61
<i>Table 2. 21. payment_transactions table</i>	61
<i>Table 2. 22. Book appointment user interface components</i>	63
<i>Table 2. 23. Book appointment used data</i>	63
<i>Table 2. 24. Schedule used data</i>	65
<i>Table 2. 25. Clear unused schedule used data</i>	67
<i>Table 2. 26. Examine user interface components</i>	69
<i>Table 2. 27. Examine used data</i>	69
<i>Table 2. 28. Join remote consultation user interface components</i>	71
<i>Table 2. 29. Join remote consultation used data</i>	71

<i>Table 2. 30. Create schedule change request user interface components</i>	<i>74</i>
<i>Table 2. 31. Create schedule change request used data</i>	<i>74</i>
<i>Table 2. 32. Handle schedule change request user interface components</i>	<i>76</i>
<i>Table 2. 33. Handle schedule change request used data</i>	<i>76</i>
<i>Table 2. 34. Pay invoices online user interface components</i>	<i>78</i>
<i>Table 2. 35. Pay invoices online used data</i>	<i>78</i>
<i>Table 2. 36. View invoices user interface components</i>	<i>81</i>
<i>Table 2. 37. View invoices used data</i>	<i>81</i>
<i>Table 2. 38. Confirm cash payment user interface components</i>	<i>82</i>
<i>Table 2. 39. Confirm cash payment used data</i>	<i>82</i>
<i>Table 3. 1. Testing risks</i>	<i>87</i>
<i>Table 3. 2. Test scenarios</i>	<i>88</i>
<i>Table 3. 3. Test cases</i>	<i>93</i>
<i>Table 3. 4. Test results</i>	<i>95</i>

LIST OF ABBREVIATIONS AND TECHNICAL TERMS

Abbreviation	Full Term	Context / Definition
ACID	Atomicity, Consistency, Isolation, Durability	A set of core properties that guarantee database transactions are processed reliably and data remains accurate even during system crashes, power cuts, or errors.
AI	Artificial Intelligence	Refers to the simulation of human intelligence processes by computer systems and machines.
AMQP	Advanced Message Queuing Protocol	An open standard application layer protocol for message-oriented middleware.
API	Application Programming Interface	A set of protocols for integration between platforms.
CRUD	Create, Read, Update, Delete	The four basic operations of persistent storage.
DoS	Denial-of-Service	A kind of cyberattack in which the perpetrator seeks to make a machine or network resource unavailable to its intended users by temporarily or indefinitely disrupting services of a host connected to a network.
EHR	Electronic Health Record	A digital, structured repository containing patient health history.
FCM	Firebase Cloud Messaging	Cross-platform cloud messaging service to distribute mobile notifications.
gRPC	gRPC Remote Procedure Calls	High-performance inter-process communication protocol in microservices.
HIS	Hospital Information System	An element of health informatics that primarily addresses the administrative and operational needs of hospitals.
HTML	HyperText Markup Language	The standard markup language for documents designed to be displayed in a web browser.
HTTP	HyperText Transfer Protocol	An application-layer protocol designed for transmitting hypermedia documents, such as HTML, across the World Wide Web.
IEEE	Institute of Electrical and Electronics Engineers	An American charitable professional organization for electrical engineering, electronics engineering, and related disciplines.

IP	Internet Protocol	The network layer communications protocol in the Internet protocol suite for relaying datagrams across network boundaries.
IPN	Instant Payment Notification	A method for online retailers to automatically track purchases and other server-to-server communication in real time.
JWT	JSON Web Token	Compact URL-safe digital signature used for secure client-side sessions.
OS	Operating System	A set of programs that control the way a computer system works.
OTP	One-Time Password	A dynamically generated password valid for a single transaction or login.
RBAC	Role-Based Access Control	An access control method that restricts network and system access based on the roles of individual users within an enterprise.
REST	Representational State Transfer	An architectural style for designing distributed hypermedia APIs.
RTI	Real-Time Interaction	Refers to the capacity to interact, exchange information, cooperate, and converse in real time without delay.
SDK	Software Development Kit	A collection of software development tools in one installable package.
SQL	Structured Query Language	A standard language for storing, manipulating and retrieving data in databases.
TCP	Transmission Control Protocol	One of the main protocols of the Internet protocol suite, providing reliable, ordered, and error-checked delivery of a stream of octets (bytes) between applications running on hosts communicating via an IP network.
UI	User Interface	Refers to the visual and interactive elements through which a user interacts with a software application, website, or digital device, including buttons, icons, typography, color schemes, and layout designs.
URL	Uniform Resource Locator	A reference to a resource on the World Wide Web.
UUID	Universally Unique Identifier	A 128-bit number used to identify information in computer systems.

UX	User Experience	Encompasses the overall experience a user has when interacting with a product, system, or service, focusing on usability, accessibility, efficiency, and emotional satisfaction.
WSL	Windows Subsystem for Linux	Compiles full native Linux systems to execute inside host Windows OS.
XSS	Cross-site Scripting	A type of security vulnerability that can be found in some web applications.

TÓM TẮT

Các hệ thống y tế hiện đại đang phải đối mặt với tình trạng tắc nghẽn trong khâu vận hành, đặc biệt là tại các phòng khám tư nhân quy mô vừa và nhỏ, nơi vẫn phụ thuộc nặng nề vào các quy trình làm việc thủ công trên giấy tờ. Việc tổ chức xếp hàng thủ công này dẫn đến tình trạng quá tải nghiêm trọng tại các phòng chờ, trong khi sự thiếu hụt hồ sơ bệnh án điện tử cản trở việc tra cứu dữ liệu nhanh chóng. Hơn nữa, các phòng khám hiện đang hoạt động dưới những giới hạn về không gian vật lý, khiến cho việc tư vấn y tế từ xa tức thời trở nên bất khả thi. Để giải quyết những thách thức này, luận văn đề xuất một nền tảng quản lý y tế kỹ thuật số toàn diện, tích hợp các chức năng đặt lịch khám, quản lý Hồ sơ bệnh án điện tử (EHR) và tư vấn y tế từ xa (telehealth).

Hệ thống máy chủ (backend) được xây dựng dựa trên kiến trúc vi dịch vụ (microservices) được container hóa với khả năng mở rộng cao, sử dụng TypeScript, NestJS và hệ quản trị cơ sở dữ liệu MySQL, cùng nền tảng Docker chạy trên môi trường Windows Subsystem for Linux (WSL). Để đạt hiệu năng và độ tin cậy tối đa, kiến trúc này tích hợp lớp bộ nhớ đệm (caching) Redis để hỗ trợ danh sách trắng (whitelist) JSON Web Token (JWT) phi trạng thái, hệ thống điều phối thông điệp (message broker) RabbitMQ cho giao tiếp liên dịch vụ bất đồng bộ, và Firebase Cloud Messaging (FCM) kết hợp cùng dịch vụ Gmail để phân phối thông báo trên thiết bị di động cũng như email xác thực OTP. Về phía giao diện người dùng (front-end), một cổng quản trị web tương thích (responsive) được phát triển bằng ReactJS nhằm xử lý các luồng công việc của phòng khám, trong khi một ứng dụng Android hiệu năng cao được xây dựng bằng Flutter để kết nối bệnh nhân và bác sĩ. Nền tảng cũng tích hợp VNPay cho các giao dịch kỹ thuật số an toàn và ZegoCloud SDK để hỗ trợ gọi video tư vấn y tế từ xa theo thời gian thực. Nguyên mẫu hệ thống microservices này hoạt động ổn định với giao tiếp API có độ trễ thấp, tính năng thông báo bất đồng bộ đáng tin cậy và các luồng truyền dữ liệu đầu cuối (end-to-end) có tính bảo mật cao, qua đó mang đến một giải pháp nâng cấp kỹ thuật số thực tiễn và mạnh mẽ cho các phòng khám tư nhân.

ABSTRACT

Modern healthcare systems suffer from operational bottlenecks, particularly in small-to-medium private clinics that heavily rely on traditional, paper-based workflows. These manual queuing structures lead to severe waiting-room overcrowding, while the lack of electronic medical records hinders quick data retrieval. Furthermore, clinics operate under restrictive physical boundaries, making immediate remote consultation inaccessible. To address these challenges, this thesis introduces a comprehensive digital healthcare management platform integrating appointment scheduling, Electronic Health Record (EHR) management, and telehealth consulting.

The system's backend is structured upon a highly scalable, containerized microservices architecture utilizing TypeScript, NestJS, and MySQL database engines, with Docker containerization running under Windows Subsystem for Linux (WSL). For maximum performance and reliability, the architecture integrates a Redis caching layer to support stateless JSON Web Token (JWT) whitelists, a RabbitMQ message broker for asynchronous cross-service communication, and Firebase Cloud Messaging (FCM) combined with the Gmail service to distribute mobile notifications and OTP verification emails. For front-end user portals, a responsive web administration portal is developed using ReactJS to handle clinic workflows, while a high-performance native Android application is developed in Flutter to connect patients and doctors. The platform integrates VNPay for secure digital transactions and ZegoCloud SDK for real-time telehealth video calling. The resulting microservices prototype operates stably with low-latency API communication, reliable asynchronous notifications, and highly secure end-to-end data pipelines, presenting a practical and robust digital upgrade for private clinics.

PART I: INTRODUCTION

1. Problem statement and rationale

The operational workflows within small-to-medium private healthcare clinics in Vietnam are severely constrained by traditional, paper-based manual protocols. In a highly digitalized era, a vast majority of local clinics still lack basic automated infrastructure. The primary bottleneck manifests in the appointment scheduling phase; patients must physically travel to the facility early in the morning, *bọc số* (manually draw a physical token from a queue machine), and endure lengthy waiting room queues. This outdated manual queuing system causes waiting spaces to become overcrowded, degrades the patient experience, and wastes extensive hours for both patients and clinical staff.

Simultaneously, the absence of standardized digital medical records restricts medical quality. Clinical logs, patient demographic profiles, allergies, diagnostic histories, and prescriptions are traditionally recorded in physical paper notebooks. When a patient returns to the clinic, searching and retrieving historical clinical data from stacked file archives is labor-intensive and error-prone. This critical lack of an EHR system severely impairs the speed and accuracy of medical diagnoses, prevents patient-centric data sharing, and compromises patient safety due to the potential misinterpretation of handwritten medical records.

Furthermore, private clinics are bound by strict physical and temporal limitations. Clinic consultations are held strictly within standard business hours and require physical presence. Patients residing in remote or rural locations face significant logistical barriers to travel for brief medical advice or non-emergency follow-ups. In situations requiring urgent or after-hours consultation, patients must either postpone care or seek overcrowded tertiary hospitals, putting additional stress on the public health infrastructure. To address these systemic operational inefficiencies and geographical barriers, there is an urgent research and engineering need to architect, implement, and validate a centralized, comprehensive digital healthcare platform incorporating automated scheduling, structured electronic health records, and low-latency virtual consults.

2. Existing solutions and prior works

In the domain of Vietnamese digital healthcare, several platforms have emerged to tackle individual components of clinic operations. Well-known national platforms like BookingCare address the scheduling bottleneck by establishing online queues across various medical networks. While BookingCare successfully reduces physical queues, it remains a centralized booking directory rather than an integrated operational system. Consequently, small private clinics find it difficult to fully customize and

synchronize BookingCare's workflows with their own internal patient tracking databases, resulting in fragmented patient histories.

Conversely, platforms such as eDoctor have introduced virtual medical consulting and basic electronic health record systems on mobile devices. Although eDoctor connects doctors and patients effectively, it operates as a direct B2C consulting marketplace. Small private clinics cannot easily adopt it as an independent clinic management tool to run their daily in-house processes. Internationally, open-source HIS frameworks like Bahmni offer complete, modular medical record keeping and scheduling. However, Bahmni's massive architecture is overly heavy and complex for small private clinics to self-host and customize. More importantly, it lacks tailored localization for Vietnamese banking structures (such as VNPay) and localized notification templates.

This research bridges these gaps by synthesizing the scheduling strengths of BookingCare with the telemedicine capabilities of eDoctor and the robust record-keeping philosophies of Bahmni. The result is a lightweight, loosely coupled microservices system tailored specifically for small private clinics, fully localized to Vietnam's payment, authentication, and communication standards.

3. Research objectives

The primary objective of this thesis is to design, develop, and deploy a comprehensive, digitalized healthcare management and online medical consulting software system tailored specifically for small-to-medium private clinics. By transforming traditional clinical workflows, the system aims to resolve critical inefficiencies such as overcrowded waiting areas caused by manual queuing, and the lack of historical medical data retrieval stemming from paper-based record-keeping. Through the integration of cross-platform client applications and secure real-time communication protocols, the project seeks to broaden patient accessibility to timely, remote healthcare services and remote medical advice during and beyond standard operating hours.

To achieve this overarching goal, the research focuses first on designing a highly secure, ACID-compliant relational database architecture using MySQL to safely store clinical operations data, appointment schedules, and EHRs. Building upon this structured data layer, a scalable backend system is developed utilizing a NestJS microservices architecture written in TypeScript. This backend acts as the core transactional engine, exposing robust and efficient RESTful APIs to orchestrate seamless communication and data exchange between the web and mobile client platforms.

On the client side, the objective involves building two distinct, specialized frontends that communicate independently with the backend microservices layer. A responsive web administration portal is developed using ReactJS to empower clinic

administrators and nursing staff to manage patient queues, doctor schedules, and daily clinic workflows smoothly. Concurrently, a high-performance native Android application is engineered using Flutter to serve as a unified portal for both doctors and patients, incorporating core features like appointment scheduling, medical history tracking, and remote video consultations.

Finally, the project aims to optimize system performance, security, and utility through third-party integrations and containerized infrastructure management. The implementation incorporates VNPay as a secure digital payment gateway for cashless appointment transactions and ZegoCloud SDK to power low-latency telehealth video consulting. System security is reinforced with a stateless JWT authentication mechanism paired with a Redis whitelist. To guarantee operational stability and prevent environment conflicts, the core services – including MySQL, Redis, and RabbitMQ – are containerized using Docker, with asynchronous message queuing handling system logging via Winston, OTP verification through Gmail, and push notifications via FCM.

4. Research subject and scope

4.1. Research subject

The primary research subject of this thesis is the process of digitizing and automating healthcare administrative workflows, medical queuing, and remote medical consultations within small-to-medium private clinics. This involves analyzing how patients interact with clinic scheduling, how medical staff manage patient flows, and how physicians retrieve electronic health records during consultations. The research specifically examines the optimization of these processes through the lens of modern software engineering paradigms, focusing on minimizing administrative bottlenecks, reducing wait times, and improving the accessibility of remote healthcare services.

In terms of technical execution, the research focuses on the design, development, and orchestration of a multi-tiered, cross-platform software system. This encompasses a strongly-typed TypeScript and NestJS microservices backend, a relational database model built on MySQL, an in-memory Redis caching layer, and an asynchronous RabbitMQ message queue. The research also investigates the implementation of front-end applications, specifically a responsive ReactJS web administration portal for clinical staff and a native Flutter mobile application serving both doctors and patients, ensuring seamless real-time data synchronization between the decoupled client platforms and the core backend services.

Furthermore, the subject includes the integration of secure third-party platforms to extend the system's core capabilities. This involves evaluating the feasibility and security of implementing the VNPay gateway to process cashless online payments for appointment bookings and the ZegoCloud SDK to facilitate low-latency, real-time video consultations. Additionally, the research explores robust cross-platform security

mechanisms, including stateless JWT authentication paired with Redis whitelisting, and secure storage techniques such as HttpOnly cookies on the web and Flutter Secure Storage on mobile devices.

4.2. Research scope

The functional scope of this thesis is strictly bounded by the core administrative and telehealth operations of a private clinic, excluding pharmacy inventory tracking, physical medication warehousing, and hardware-level network optimizations. The development is focused on delivering a functional scheduling module with automated queuing, a digital EHR database, an integrated online payment gateway via VNPay, and a RTI-based real-time video consulting module via ZegoCloud. The system also integrates asynchronous background processes, limiting its messaging boundaries to Gmail for OTP verifications, Winston for system logging, and FCM for push notifications.

Technically, the development and deployment scope are limited to a containerized microservices environment hosted locally. The infrastructure leverages Docker combined with the WSL to isolate and host the MySQL database, Redis caching layers, and RabbitMQ message broker, thereby eliminating environment conflicts and ensuring consistency. The mobile client application is designed and compiled specifically for the Android operating system using Flutter, while the clinic administration portal is developed as a responsive single-page web application using ReactJS.

The data scope for testing and system verification relies entirely on self-generated mock datasets to protect actual patient privacy. This data includes synthesized demographic profiles, simulated clinical histories, mock appointment schedules, and standardized catalogs of diseases and medications used solely to link diagnoses to patient charts. Finally, the methodological scope of the project follows the linear, sequential phases of the Waterfall model, spanning initial requirement specifications, system architectural and physical database design, full-stack software implementation, rigorous RESTful API testing via Postman, and cross-platform performance evaluation.

5. Research Content and Technologies Used

a) Research content

- Analyzing traditional clinical administrative workflows, manual queuing inefficiencies, and electronic health record-keeping limitations in small-to-medium private clinics
- Investigating modern software architecture paradigms, specifically containerized microservices architectures, stateless session management, and distributed caching, to address system scalability and security

- Analyzing and drafting functional requirements, system specifications, and detailed use case scenarios for clinical scheduling, queue management, EHR synchronization, and remote video consultations
- Designing a secure, fully normalized relational database model using MySQL to handle electronic medical records, patient credentials, appointment slots, and clinical metadata
- Developing a strongly-typed TypeScript and NestJS-based microservices backend to serve as the core transactional engine, exposing secure RESTful APIs to client platforms
- Implementing a highly responsive web-based clinical administration portal using ReactJS to empower clinic staff to manage daily queues, doctor schedules, and billing workflows
- Engineering a cross-platform mobile application using Flutter to serve as a unified portal for both doctors and patients, incorporating appointment booking, digital EHR tracking, and telehealth modules
- Integrating secure payment processing via the VNPay gateway and low-latency, real-time audio-video streams via ZegoCloud SDK to facilitate virtual healthcare consultations
- Conducting comprehensive unit testing, functional verification, and API integration testing using Postman to ensure seamless communication between client applications and backend microservices
- Deploying the containerized microservices infrastructure locally using Docker combined with WSL to guarantee environment consistency and system stability

b) Technologies used

- **Backend services:**
 - NestJS (TypeScript): Employed as the primary progressive Node.js framework for structuring robust, scalable, and loosely-coupled backend microservices
 - RabbitMQ: Implemented as an asynchronous message broker to handle non-blocking communications, system logs, push notifications, and email dispatches across services
 - gRPC: Utilized for high-performance, low-overhead, and synchronous inter-service communications within the backend microservices layer
- **Frontend interfaces:**
 - ReactJS: Adopted to build a responsive, component-based, single-page web administration portal for clinical staff and receptionists
 - Flutter (Dart): Used to develop a high-performance, native-like mobile application deployed on Android for both doctors and patient portals
- **Data storage and caching:**
 - MySQL: Deployed as the primary relational database management system to guarantee strict ACID transactions for sensitive electronic health records and booking invoices

- Redis: Integrated as an in-memory database to store the JWT whitelist for stateless session control and caching operational data
- **Telehealth and payment integrations:**
 - ZegoCloud SDK: Embedded to establish secure, low-latency, real-time video and audio streams for virtual consulting between patients and physicians
 - VNPay Sandbox Gateway: Integrated to facilitate cashless transaction processing, digital invoicing, and automatic appointment reservation confirmation
- **Infrastructure, logging and automation:**
 - Docker and WSL: Strategically utilized to containerize, orchestrate, and isolate all core system databases and message brokers, ensuring environment consistency
 - FCM: Implemented to deliver instantaneous, cross-platform push notifications regarding appointment updates to the Flutter mobile application
 - Winston and Postman: Used for centralized, structured file logging and automated RESTful API testing, verification, and debugging

6. Contributions of the thesis

- Healthcare, operational, and social contributions:

The developed system directly modernizes administrative and medical operations for small-to-medium private clinics by replacing traditional, manual paper-based processes with an automated, digitalized workflow. By optimizing patient registration, clinical scheduling, and EHR management, the platform effectively reduces patient wait times, prevents overcrowding in clinic waiting areas, and enhances overall clinical operational efficiency. Furthermore, the integration of a centralized, secure EHR subsystem empowers physicians with instant, chronological access to historical patient health data, facilitating faster and more accurate clinical diagnoses while providing patients with a secure mobile portal to track their personal medical histories and follow up on treatments systematically.

From a patient-centric and social standpoint, the system breaks down geographical and operational barriers by introducing a low-latency, RTI-based telehealth video consulting module. Coupled with secure cashless transaction processing via the VNPay payment gateway, the platform enables patients – especially those in remote areas or with limited mobility – to access timely, professional medical consultations and advice beyond standard physical clinic hours. This digital integration significantly improves the quality of care and enhances the overall patient experience in private practice settings.

- Professional training and development contributions:

Through the practical design and execution of this thesis, the researcher has consolidated advanced software engineering competencies, particularly in architecting and orchestrating a distributed, containerized microservices backend. The project provided rigorous hands-on experience in managing loosely coupled services, synchronous inter-service communication via gRPC, and asynchronous message broker routing using RabbitMQ. Additionally, the development lifecycle facilitated a deep practical understanding of multi-platform engineering, including building highly responsive, component-based single-page web applications with ReactJS for administrative staff, and native Android mobile applications with Flutter to deliver a seamless, high-performance user experience for both doctors and patients.

Furthermore, the project advanced the researcher's skills in implementing robust security and DevOps practices, such as stateless session control utilizing JWT with a Redis-backed whitelist, secure client-side storage mechanisms, and containerized deployment. The database, caching, and message broker services were containerized using Docker under the WSL, ensuring environment consistency and high system reliability. Beyond serving as a practical clinic management prototype, this system provides a highly valuable technical reference and practical demonstration of modern software engineering methodologies for future developments in digital healthcare.

7. Thesis organization

The main body of this graduation thesis is systematically structured into three major parts: Introduction, Content, and Conclusion, alongside a dedicated reference list and supplementary appendices.

- **Introduction:**

This introductory section establishes the foundation of the research. It presents the background and the problem statement regarding manual clinical workflows, reviews existing telehealth and scheduling solutions, and defines the general and specific research objectives. Furthermore, this part delineates the subject, scope, and limitations of the thesis, details the software development content and the technologies employed, highlights the key social and technical contributions, and outlines this overall thesis organization.

- **Content:**

Problem description: This chapter provides a detailed analysis of traditional clinical administrative processes, booking queues, and electronic health record management. It specifies the system's use case diagrams and specifications for both clinic administrators and end-users (doctors and patients), accompanied by non-functional performance, security requirements, and clinical business rules.

Architecture design and data modeling: This chapter describes the technical design of the proposed clinical platform. It details the multi-tier containerized

microservices architecture utilizing NestJS, gRPC, and RabbitMQ, constructs the normalized MySQL relational database schema and data dictionary, configures stateless JWT session management with Redis whitelisting, and outlines the physical integration models for VNPay payment gateways and ZegoCloud telehealth streams.

Software testing and performance evaluation: This chapter presents the testing objectives and implementation methodologies. It details the systematic test plan, functional test scenarios, API testing logs conducted via Postman, and client-side performance evaluation on ReactJS and Flutter applications, summarizing the verified test results and system stability.

- **Conclusion:**

This final section summarizes the entire research project. It evaluates the achieved results against the initial objectives, addresses the practical challenges and technical limitations encountered during development, and proposes concrete directions for future development – such as orchestrating services with Kubernetes, optimizing data synchronization, or incorporating smart healthcare analytics.

- **References:**

This section compiles a comprehensive bibliography of all academic publications, technical standards, and framework documentations referenced throughout the thesis. All entries are systematically managed and formatted in strict accordance with the IEEE citation style.

PART II: CONTENT

CHAPTER 1: PROBLEM DESCRIPTION

1. Detailed problem description

The operational paradigm of traditional small-to-medium private clinics is heavily restricted by legacy, manual processes that compromise both administrative efficiency and the quality of patient care. Under the conventional workflow, the patient journey begins with physical arrival at the clinic, where patients must wait in line at the reception desk to manually register their information in paper logbooks. A receptionist then issues a physical paper ticket representing their sequence in the queue. This manual queuing mechanism lacks real-time updates and scheduling visibility, meaning patients are forced to wait indefinitely in crowded waiting areas without knowing when they will be examined. Consequently, clinics regularly face severe bottlenecks during peak hours, resulting in operational chaos, high administrative overhead for nursing staff, and a significantly diminished patient experience.

To overcome these operational challenges, the proposed system introduces a secure, unified digital healthcare platform that digitalizes and orchestrates the workflows of small-to-medium practices. The digitized patient journey begins on a native Flutter mobile application. To schedule an appointment, patients select their desired medical department, date of examination, and the preferred consultation method, which can be either an in-person physical visit (offline) or a remote telehealth consultation (online). Once these parameters are specified, the system queries the backend to retrieve real-time schedule vacancies from the database and dynamically displays a list of eligible doctors alongside their available working shifts for the patient to choose from. Additionally, this mobile portal empowers patients to securely track their own medical history, review historical clinical diagnoses, access digital prescriptions, and monitor their previous appointment schedules.

A major architectural refinement of this platform is the post-consultation payment and checkout workflow, designed to seamlessly synchronize digital transactions with physical clinic operations. Unlike typical systems requiring upfront prepayment, patients complete their examination fees after receiving their medical consultation. Immediately upon the doctor completing the medical examination and entering the clinical diagnosis into the EHR database, the system automatically generates a unique digital invoice containing the pre-calculated consultation fee and the prescribed medication list. This flexible approach accommodates both cashless digital transactions and traditional cash payments against this pre-existing invoice. For online transactions, a new VNPay transaction reference (txn_ref) is dynamically generated each time the patient initiates a payment session through the integrated VNPay sandbox gateway. For cash transactions, patients pay physically at the clinic,

where the receptionist or nurse retrieves the pre-existing invoice and manually confirms the payment status through the web administration portal. Once the payment is successfully verified and the invoice status is updated – either via the VNPay payment webhook or manual clinical confirmation – the patient presents the invoice code to the nurse, who validates the transaction to dispense the prescribed medications and explain usage instructions.

The division of labor among clinical stakeholders is clearly defined through specialized client interfaces designed to optimize their respective tasks. On the web-based clinical administration portal built with ReactJS, the nurse's primary responsibilities are streamlined to focus on confirming cash payments, retrieving detailed digital invoices for pharmaceutical dispensing, and providing patients with follow-up medication instructions. Meanwhile, physicians access their workflows via the Flutter mobile application. Doctors can continuously access and review the comprehensive EHRs of their upcoming patients, ensuring thorough clinical preparation. The doctor's diagnostic interface remains highly consistent across consultation types; the only interface variation is a dedicated button to join a secure, low-latency RTI video conference call powered by ZegoCloud SDK for online sessions. Once the consultation is complete, doctors enter diagnostic findings, clinical notes, and prescriptions into the centralized MySQL database using an identical, standardized form for both online and offline patients.

On the backend, a scalable NestJS microservices architecture written in TypeScript orchestrates these operations with optimized communication and notification flows. To balance system responsiveness and reliability, communication tasks are strictly segregated into synchronous and asynchronous pipelines. The dispatch of OTP verification emails via the Gmail service is executed as a synchronous transaction to guarantee instant, real-time security validation for user authentication. Conversely, non-critical push notifications – including clinical schedule updates, booking confirmations, and queue alerts – are treated as asynchronous events. These notifications are dispatched to the RabbitMQ message broker, which queues and processes them asynchronously before pushing them to the Flutter client via FCM. This asynchronous decoupling prevents main thread blocking, reduces API gateway latency, and ensures high system performance even during periods of heavy clinic traffic.

2. Functional requirements

2.1. Use case diagrams

2.1.1. General use case diagram

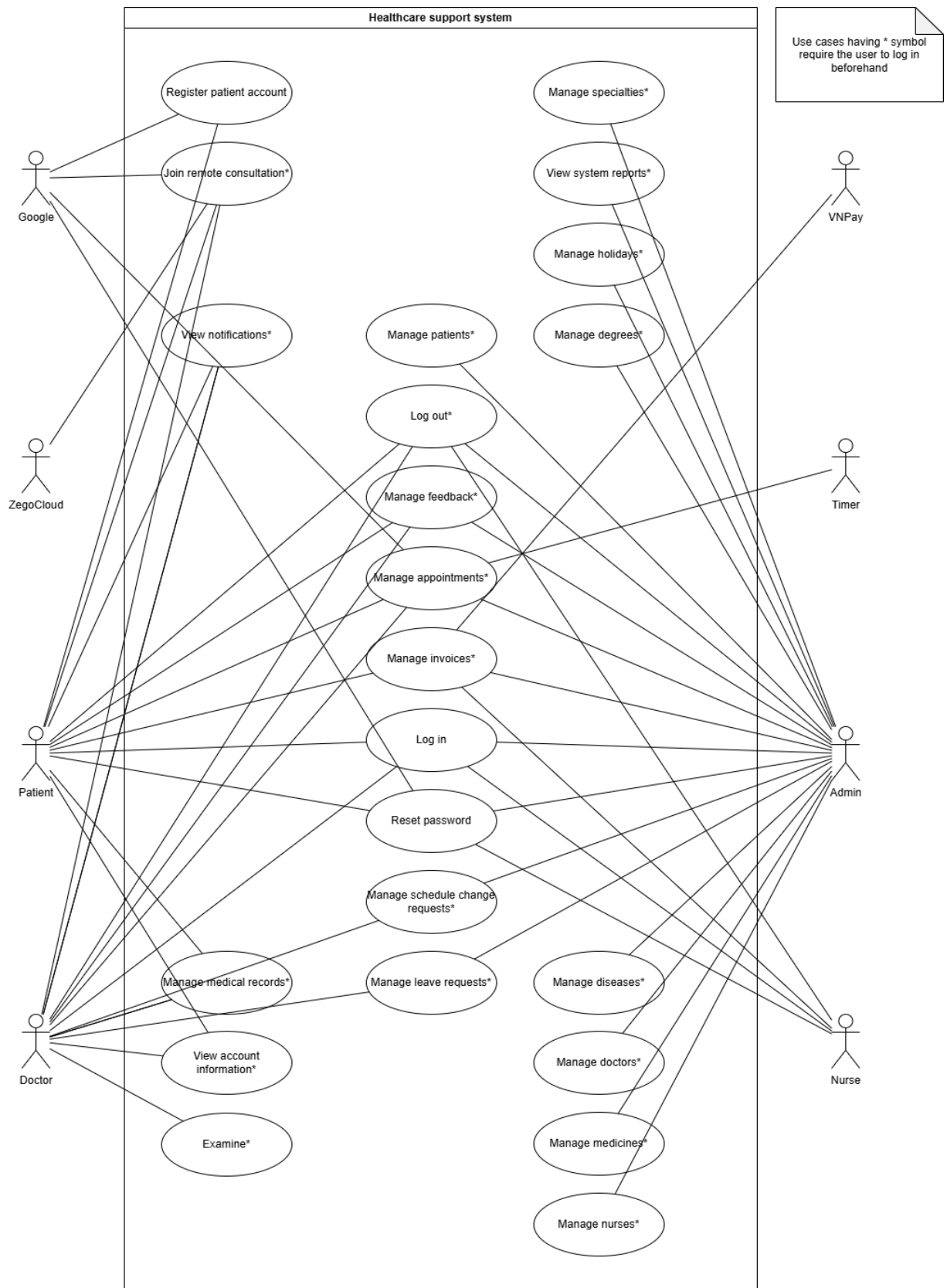


Figure 1. 1. General use case diagram

Picture 1.1 illustrates the simplified usecase diagram of the clinic management

and telehealth support system. This diagram visually delineates the core functional modules and the interactive relationships between four human actors (Patient, Doctor, Nurse, and Admin), one internal system actor (Timer) and three external system actors (VNPAY, ZegoCloud and Google) within the boundaries of the digital healthcare platform. The system is systematically organized into key functional areas: Account management (comprising registration, login, logout, and password recovery), clinical appointment scheduling (manage appointments), billing and payment processing (manage invoices), electronic health record management (manage medical records and examine), remote telehealth consultation (join remote consultation), and advanced master database administration subsystems reserved exclusively for the admin. Notably, to optimize clinical data security and strictly enforce RBAC, the majority of the usecases within the diagram are designated with an asterisk (*) symbol, representing a mandatory precondition where users must successfully authenticate via the login process to establish a valid and authorized session prior to execution.

2.1.2. Patient actor use case diagram

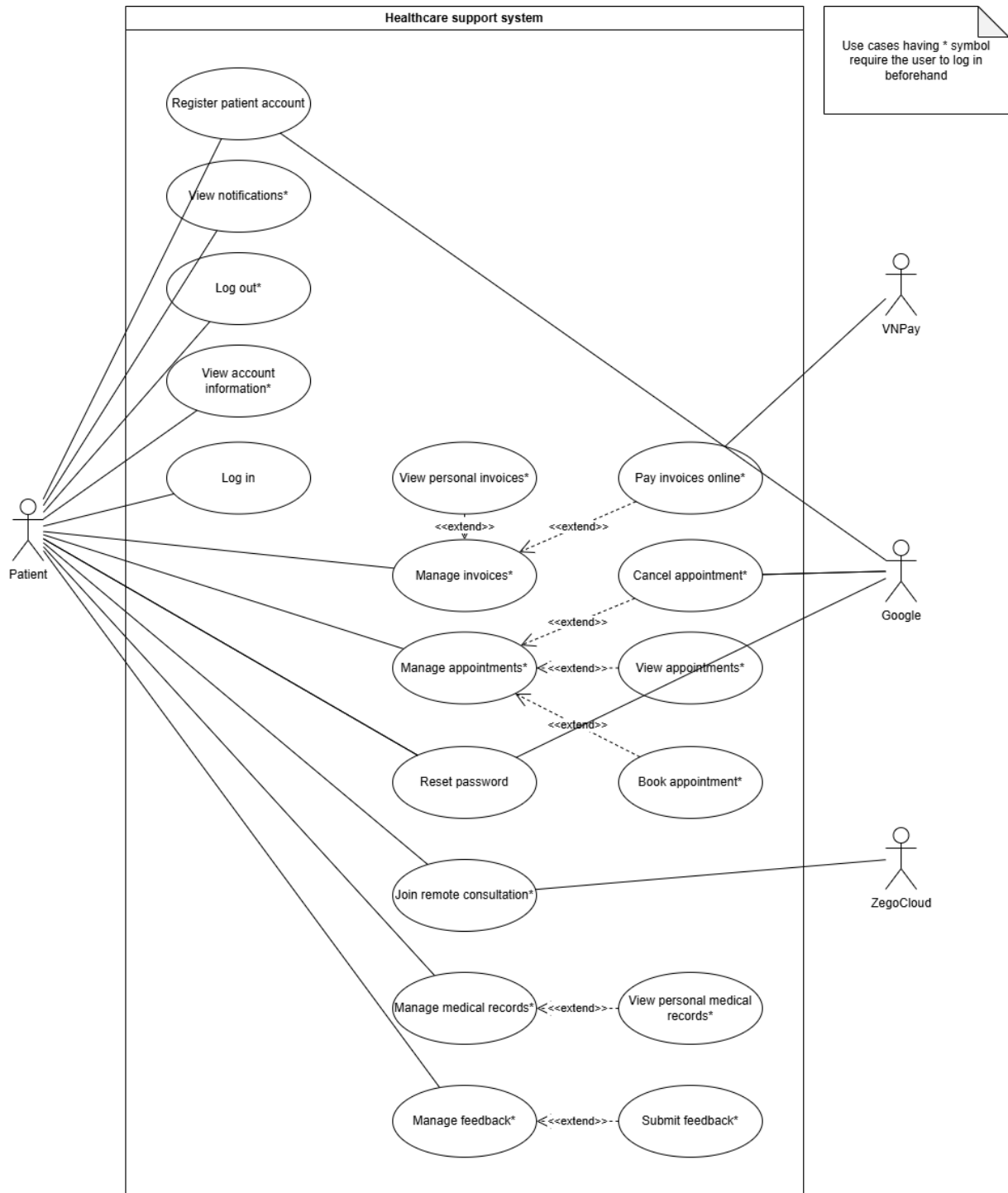


Figure 1. 2. Patient actor use case diagram

Figure 1.2 details the use case diagram for the Patient actor, who primarily interacts with the system via the cross-platform Flutter mobile application to manage personal healthcare services. The Patient's functional journey is divided into several logically organized modules: Account management, appointment coordination, financial operations, clinical record access, and telehealth engagement. Within account management, the patient can perform log in, reset password, and register patient account, which synchronously interfaces with the external Google (Gmail SMTP)

service to dispatch a secure OTP for email verification, alongside authenticated actions like Log out and View account information. For clinical appointments, the base use case Manage appointments is extended by Book appointment (allowing patients to search schedules, select departments, dates, and doctors), View appointments, and Cancel appointment. In Financial Operations, post-consultation billing is handled through Manage invoices, which is extended by View personal invoices and Pay invoices online, with the latter seamlessly interfacing with the third-party VNPay gateway to process cashless digital transactions. Furthermore, patients retrieve historical EHRs and prescriptions via the Manage medical records base use case, which is extended by View personal medical records, while virtual video consultations are initiated through the Join remote consultation use case that integrates directly with ZegoCloud RTI services to establish low-latency audio-video streams. Finally, patients receive clinic alerts through the View notifications use case and provide feedback via Manage feedback (extended by Submit feedback), with all asterisk-tagged (*) use cases strictly enforcing RBAC by requiring successful user authentication beforehand.

2.1.3. Doctor actor use case diagram

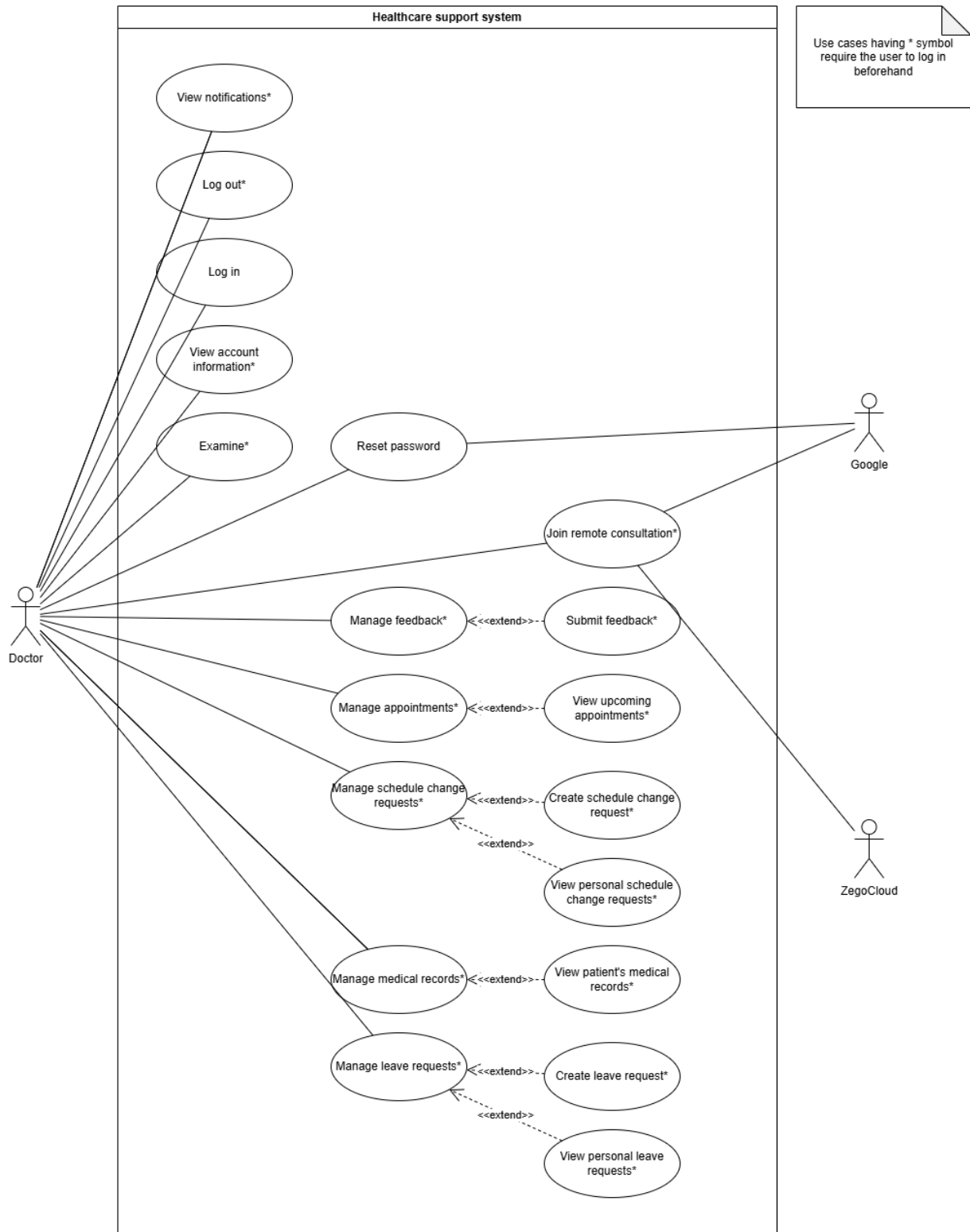


Figure 1. 3. Doctor actor use case diagram

Figure 1.3 details the use case diagram for the Doctor actor, who primarily interacts with the system via the native Flutter mobile application to manage clinical duties, schedule modifications, and telehealth sessions. The doctor's functional journey is systematically organized into several key operational areas: Account settings, clinical Consultations, schedule and leave adjustments, and Electronic Health Record

Access. Within account settings, the doctor can perform Log in, Log out, View account information, Reset password and View notifications. The core clinical examination workflow is represented by the Examine use case, where the doctor conducts diagnoses and enters digital prescriptions. For remote consultations, the doctor executes the Join remote consultation use case, which integrates directly with the external ZegoCloud platform to establish low-latency, real-time RTI audio-video streaming. Under schedule and leave operations, the base usecase Manage appointments is extended by View upcoming appointments, while Manage schedule change requests is extended by Create schedule change request and View personal schedule change requests to allow flexible roster adjustments. Concurrently, the Doctor coordinates their planned absences using the Manage leave requests base use case, which is extended by Create leave request and View personal leave requests to systematically submit and track time-off applications. Additionally, the Doctor manages patient histories via the Manage medical records base use case, which is extended by View patient's medical records to securely retrieve historical EHRs. Finally, the Doctor can interact with the clinic administration through Manage feedback (extended by Submit feedback), with all asterisk-tagged (*) use cases strictly requiring successful user authentication beforehand to enforce RBAC and protect sensitive medical databases.

2.1.4. Admin and Timer actor use case diagram

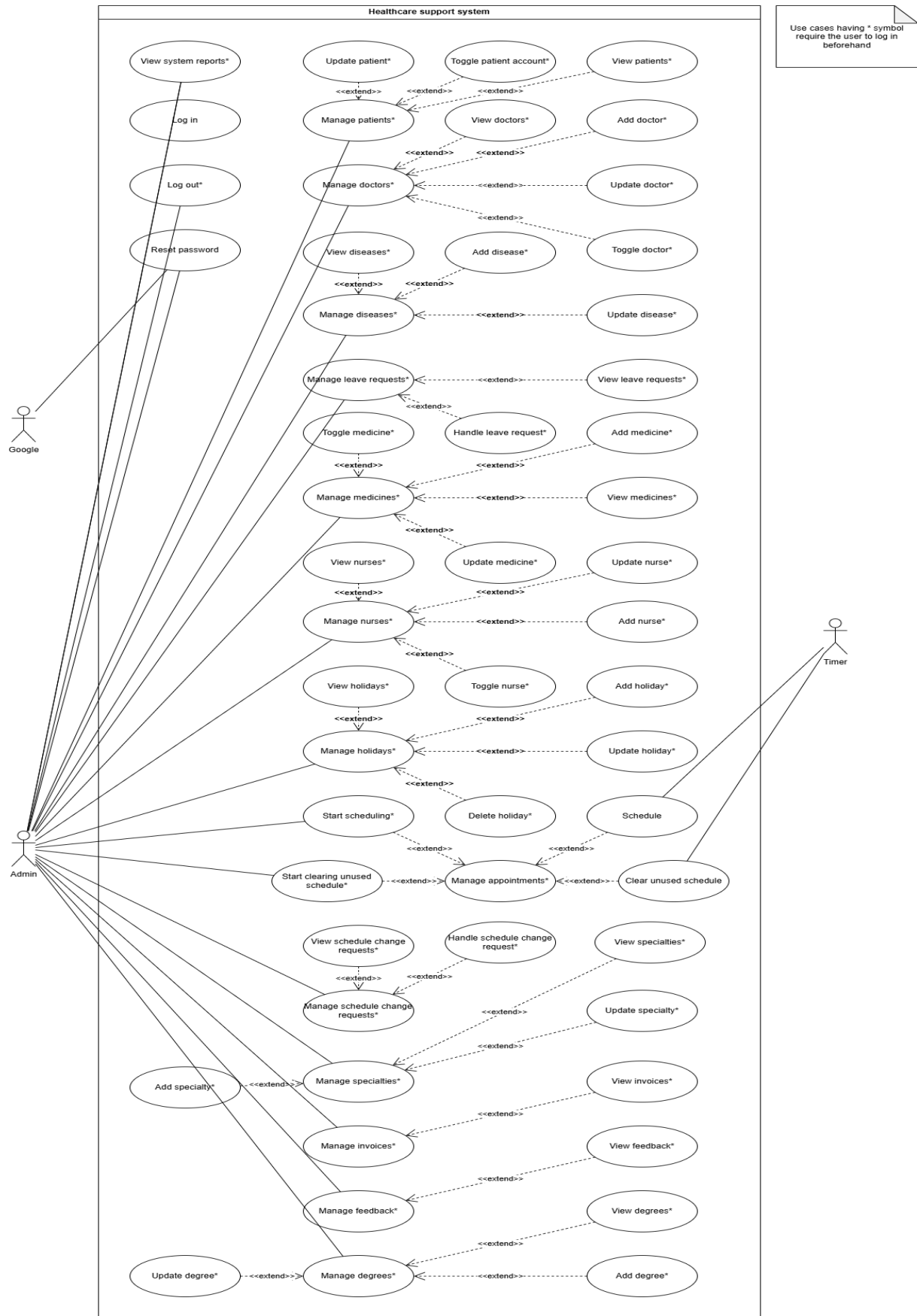


Figure 1. 4. Admin and Timer actor use case diagram

Figure 1.4 details the use case diagram for the Admin and Timer actors, showcasing the administrative management of clinical metadata and the orchestration of automated server-side operations. The Admin represents the privileged administrative user who interacts with the system via the ReactJS web portal to maintain structural catalogs, user roles, and clinical master data. This comprehensive scope of administrative duties is systematically decoupled into logical base use cases that are further extended by specialized CRUD operations, including: Manage patients (extended by View patients, Update patient, and Toggle patient account), Manage doctors (extended by View doctors, Add doctor, Update doctor, and Toggle doctor), Manage diseases (extended by View diseases, Add disease, and Update disease), Manage leave requests (extended by View leave requests and Handle leave request), Manage medicines (extended by View medicines, Add medicine, Update medicine, and Toggle medicine), Manage nurses (extended by View nurses, Add nurse, Update nurse, and Toggle nurse), and Manage holidays (extended by View holidays, Add holiday, Update holiday, and Delete holiday). The Admin also coordinates clinical entities via Manage specialties (extended by View specialties, Add specialty, and Update specialty), Manage degrees (extended by View degrees, Add degree, and Update degree), Manage schedule change requests (extended by View schedule change requests and Handle schedule change request), Manage invoices (extended by View invoices), and Manage feedback (extended by View feedback). Crucially, the Timer actor represents an internal server-side scheduler counter designed to run asynchronous background processes and automated Cron jobs. Rather than interacting as an external third-party service, the Timer directly triggers Schedule (which extends the base Start scheduling use case to automatically generate doctor shifts) and Clear unused schedule (which extends Manage appointments to automatically release expired booking slots). In alignment with the system's security architecture, all administrative use cases designated with an asterisk (*) strictly enforce RBAC, requiring successful authentication beforehand to protect the database against unauthorized master catalog modifications.

2.1.5. Nurse actor use case diagram

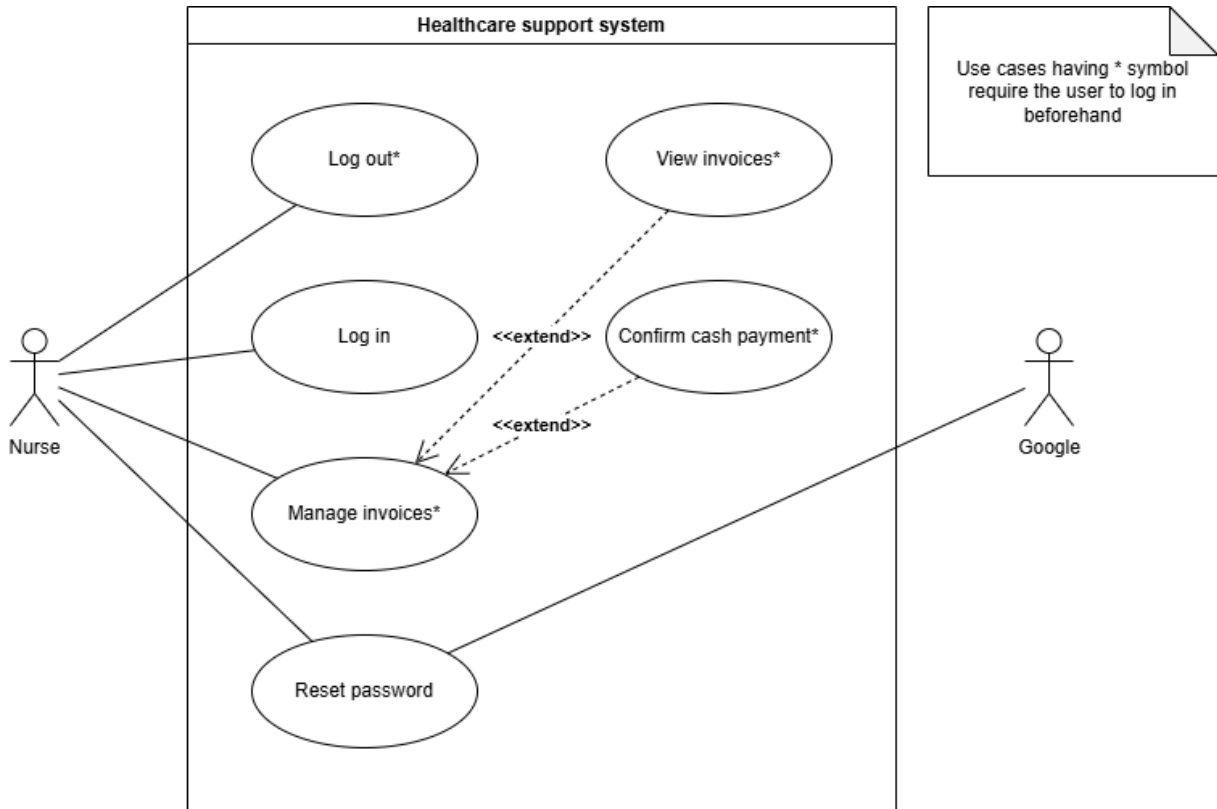


Figure 1. 5. Nurse actor use case diagram

Figure 1.5 details the use case diagram for the Nurse actor, who primarily interacts with the system via the responsive ReactJS web administration portal to coordinate clinical checkout workflows and in-person transactions. To ensure high clinical throughput and strict separation of privileges, the Nurse's operational boundary is intentionally streamlined and restricted from schedule or medical catalog modifications, focusing solely on account settings and billing management. Under account settings, the Nurse can execute the Log in, Reset password, and authenticated Log out use cases, with password recovery interfacing with the external Google (Gmail SMTP) service to securely dispatch verification codes. The core of the Nurse's clinical contribution is orchestrated under the Manage invoices base use case, which is structurally extended by View invoices – enabling the nurse to retrieve detailed billing records and check attached prescriptions for accurate drug dispensing and patient counseling – and Confirm cash payment – which allows the manual verification and status update of physical cash transactions at the clinic's reception. In strict compliance with the system's RBAC and data protection standards, all asterisk-tagged (*) use cases require the Nurse to possess a valid, authenticated session, thereby safeguarding financial logs and sensitive medical data against unauthorized access.

2.2. Use case specifications

2.2.1. Book appointment

Use case name: Book appointment		ID: UC-01
Actors: Patient, Google		Priority: High
		Complexity: High
Participants and concerns: - Patient: Needs an easy booking process, the ability to select the correct available time slot matching their schedule, a successful booking confirmation, and assurance against double-booking conflicts - Doctor: Wants real-time updates on new patient appointments (via Push Notification) to stay informed and prepare for the consultation session - System: Accurately manages appointments, strictly avoids time slot duplication (race conditions), prevents users from spamming excessive virtual bookings, and restricts patients with overdue, unpaid examination bills		
Brief description: The patient selects an available time slot of a doctor to make an appointment. Before permitting the booking, the system executes business barriers: Checking unpaid invoices and the maximum active appointment limit. If requirements are met, the system creates booking data, updates the time slot status, and simultaneously emits a notification event.		
Relationships: - Association: Patient - Extend: Manage appointments - Include: None		
Normal flow of events:	1. The Patient enters the department, service type, and appointment date, views doctor information, and selects an available time slot on the Mobile App interface. 2. The patient confirms the "Book Appointment" request. The app sends a request to the API Gateway. 3. The API Gateway authenticates the Token and checks permissions (ensuring the sender is a Patient). 4. The system (Schedule Service) calls gRPC to the Payment Service to count the number of unpaid invoices for this patient. If the count is 0, the patient passes the financial check. 5. The system opens a Database Transaction. 6. The system queries and counts the number of future appointments (from today onwards) currently in CONFIRMED status belonging to the Patient using a read lock mode (pessimistic_read). The patient satisfies the condition (Fewer than 3 slots). 7. The system checks the timeSlotId. Ensures that the time slot exists, is available (AVAILABLE), and has not	

	passed its start time. This time slot table row will be write-locked (pessimistic_write) to prevent others from manipulating it simultaneously 8. The system saves the new Booking record with a CONFIRMED status. 9. Updates the corresponding Time Slot record to the booked status (BOOKED). 10. Commits the Transaction to permanently persist the data. 11. The system emits a booking_created event via RabbitMQ. The Notification Service catches this event and sends a Push Notification to the Doctor's phone. Returns a success result to the Patient
Branching / Alternative flows:	Sub1. Patient Has Unpaid Debts (At Step 4): 1. The Payment Service reports at least one invoice with status = UNPAID. 2. The system rejects immediately, returning a 409 Conflict error: "Patient has unpaid invoices." Sub 2. Patient Reaches Maximum Booking Threshold (At Step 6): 1. The system counts that the patient currently holds 3 or more uncompleted appointments. 2. Returns a 409 Conflict error: "Patient can only have a maximum of 3 active appointments at the same time." Sub 3. Time Slot Already Booked by Someone Else (At Step 7): 1. The current time slot status is BOOKED (grabbed by someone else milliseconds prior). 2. Returns a 409 Conflict error: "This time slot has been booked or does not exist." Sub 4. Booking Window Expired (At Step 7): 1. The current time has slipped past the start time of the time slot. Returns a 409 Conflict error: "The booking window for this time slot has expired."
Exceptional / Special flows:	Sub 5. Concurrency Conflict (Race Condition): 1. If two patients simultaneously click "Book Appointment" for the same time slot at the exact same millisecond, the Database Transaction + pessimistic_write mechanism at Step 7 will only allow one flow to pass and commit successfully. 2. The later transaction will be rejected/rolled back, ensuring there are never two CONFIRMED appointments for the same Time Slot. Sub 6. Service or Database Connection Failure (System Error): 1. If a network disconnection occurs between the API

	Gateway and the Schedule Service, or if the Database server is overloaded and cannot write at Step 7. 2. The system intercepts via a try-catch block and rolls back all data states. 3. The system records the error through the Log Service. 4. Returns a 500 Internal Server Error to the Mobile App ("The system is experiencing disruptions, please try again later"). Sub 7. Active Lock Process (System Busy): 1. If precisely when the patient books an appointment, the automated Timer (Automated Weekly Scheduling Use Case) is running and holds the Lock:Scheduling. The system returns a 409 Conflict error: "System is busy scheduling, please try again later" to protect database safety.
--	---

Table 1. 1. Book appointment use case specification

2.2.2. Schedule

Use case name: Schedule	ID: UC-02
Actors: Timer	Priority: Very high
	Complexity: High
Participants and concerns: - Timer: Responsible for triggering precisely at the configured schedule (Weekly – EVERY_WEEK) - Admin: Wants all doctor schedules to be generated automatically and accurately according to template configurations without manual labor; ensures no schedules are generated on public holidays or approved doctor leave dates - Patient: Wants to consistently see available time slots for upcoming days on the application	
Brief description: The system uses an automated scheduler to periodically trigger the time slot generation process on a weekly basis. The process calculates the required time range based on doctors' weekly working templates, excludes general public holidays (GlobalHolidays) and individual doctor leaves (DoctorLeaves), and automatically initializes batches of available time slots (AVAILABLE) for the upcoming week to facilitate patient bookings.	
Relationships: - Association: Timer - Extend: Manage appointments - Include: None	
Normal flow of events:	1. The NestJS Timer (Cron Job) automatically triggers according to the preset schedule. 2. The system checks the Redis Cache (Lock:Scheduling) to

	ensure no other process is simultaneously executing this task (Distributed Lock). 3. The system sets the lock flag on Redis with a 10-minute expiration time and opens a Database Transaction. 4. The system queries the Database to retrieve the latest date for which time slots have already been generated (maxExistedDate) using a read lock mode (pessimistic_read). 5. The system determines the time frame to generate schedules: running from the consecutive day after maxDate until the end of the next Sunday. 6. The system fetches the list of Public Holidays (GlobalHolidays) and excludes these dates from the list of dates requiring schedule generation. 7. The system retrieves the Weekly Working Templates (DoctorWeeklyTemplate) and Approved Leaves (DoctorLeaves) of all doctors. 8. The system begins calculating and generating time slots: - Scans through each target date - If a doctor is on approved leave on that date, skips time slot generation for them - Otherwise, breaks down the working hours into individual time slots based on the slot duration configuration (e.g., 30 minutes = 1 slot) retrieved from the environment variable <code>TIME_SLOT_DURATION_MIN</code> 9. The system saves the entire list of generated TimeSlots into the Database with the status AVAILABLE. 10. Commits the Database Transaction to confirm successful data persistence. 11. Release the lock on Redis. The process terminates, and the system logs a success status.
Branching / Alternative flows:	Sub1. Another process is already running (At Step 2): 1. If the Lock:Scheduling key already exists in Redis (indicating another instance/server is running this process). 2. The system writes a Warning Log: "A process is already running." 3. Immediately aborts the current execution flow to prevent duplicate schedule generation. Sub 2. Schedules have already been fully generated (At Step 5): 1. If maxDate (the date with existing schedules) is greater than or equal to the next Sunday (meaning schedules were pre-generated via other means). 2. The system commits the transaction and halts the process, avoiding unnecessary schedule creation and resource

	consumption.
Exceptional / Special flows:	Sub 3. System Error, DB Disconnection, or Logic Error during Generation: 1. When the processing flow encounters any Exception/Error thrown within the try block, the system triggers the catch block. 2. All generated data will be rolled back via Transaction Rollback, ensuring the database is not left in a half-saved state (partial schedules). 3. Writes a high-severity error log (Error Log) with the error code message. 4. Always executes the finally block to guarantee the removal of the Lock:Scheduling key in Redis (preventing the system from getting "locked out" and failing to generate schedules the following week).

Table 1. 2. Schedule use case specification

2.2.3. Clear unused schedule

Use case name: Clear unused schedule		ID: UC-03
Actors: Timer		Priority: Medium
		Complexity: Low
Participants and concerns: - Timer: Ensures precise activation at 01:00 AM everyday - Admin: Concerned with server performance; wants to clean up orphaned records or unclaimed/expired time slots from the past to keep the database lightweight and compact		
Brief description: Periodically at 01:00 AM every day, the system automatically scans the database to search for past bookings and time slots. It automatically updates "stuck" bookings (those confirmed but never checked-in or completed) to a CANCELED status and permanently deletes unbooked time slots to free up storage space.		
Relationships: - Association: Timer - Extend: Manage appointments - Include: None		
Normal flow of events:	1. The Timer (Cron) triggers the automated process precisely at the configured schedule. 2. The system logs a notification: "Old schedule cleanup started.". 3. The system creates and opens a new connection to the database (Database Transaction). 4. The system executes an UPDATE query (Booking	

	cleanup): Finds all bookings linked to time slots in the past (<code>clinic_date < CURDATE()</code>) that are stuck in the <code>CONFIRMED</code> status and changes their status to <code>CANCELED</code>. (Reason: To handle cases where the patient fails to show up or the doctor forgets to click Complete, ensuring such appointments are closed rather than left hanging indefinitely). - The system executes a <code>DELETE</code> query (Unclaimed slot cleanup): Finds all past time slots that remain in an available status (<code>AVAILABLE</code>) and have no references to any Bookings (using a <code>LEFT JOIN</code> check with <code>IS NULL</code>). Physically deletes these records from the database. - The system commits the transaction to apply changes to the underlying data. - Release the connection and logs "Old schedule cleanup successful", concluding the process.
Branching / Alternative flows:	
Exceptional / Special flows:	Sub 1. Database Query Failure or Network Disconnection (System Error): - If the database server abruptly crashes halfway through – after the <code>UPDATE</code> query has executed but before the <code>DELETE</code> query runs – the try-catch block catches the error. - The system calls a Transaction Rollback. The data modified by the previous <code>UPDATE</code> query is instantly restored to its original state, maintaining database integrity. - Logs a high-severity error (Error Log): "Error while cleaning up old schedules, error: [Error Details]" for the maintenance team to inspect.

Table 1. 3. Clear unused schedule use case specification

2.2.4. Examine

Use case name: Examine	ID: UC-04
Actors: Doctor	Priority: Very high
	Complexity: Very high
Participants and concerns: - Doctor: Needs to easily input examination results, prescribe medications, and complete the appointment to proceed to the next patient - Patient: Wants medical records and prescriptions to display instantly on the app after the doctor concludes the session - Nurse: Needs invoices generated automatically and accurately on the system	

right after the examination to proceed with payment collection - System: Must ensure no invoice is generated without a medical record, and no appointment status is updated if invoice creation fails	
Brief description: Following the examination, the doctor fills in clinical indicators, diagnosis, disease codes, and medication lists, then clicks "Complete". The system executes a chained distributed transaction: Retrieves the doctor's consultation fee, saves the Medical Record & Prescription, calculates the total amount and saves the Invoice, marks the Appointment as finished. If any link in the chain breaks (fails), the system automatically cleans up any pre-created data.	
Relationships: - Association: Doctor - Extend: None - Include: None	
Normal flow of events:	1. The Doctor fills in the Medical Record and Prescription information on the Mobile App and submits the completion request. 2. The System (Schedule Service) opens a Database Transaction, queries to find the appointment (bookingId), and locks this record using pessimistic_write mode to prevent parallel modifications. 3. The System verifies the actual time: ensures that the current time has passed the start time (startTime) of the appointment. 4. [RPC Call 1]: The Schedule Service makes a gRPC call to the User Service to retrieve the consultation fee (examinationFee) of this Doctor. 5. [RPC Call 2]: The Schedule Service makes a gRPC call to the Medical Record Service, transmitting the Medical Record and Prescription details. This service processes the data, finalizes medicine pricing, stores the data, and returns the Medical Record ID (recordId) along with the total medicine fee (medicineFee). 6. The System calculates the Total Invoice Amount using the formula: $\text{totalAmount} = \text{examinationFee} + \text{medicineFee}$. 7. [RPC Call 3]: The Schedule Service makes a gRPC call to the Payment Service requesting the creation of an Invoice (Invoice) with the calculated amounts. This service successfully saves the invoice and returns the invoiceId. 8. The System updates the status of the Appointment (Booking) to FINISHED. 9. Commits the Database Transaction to persist the appointment status change, concluding the entire Saga process. 10. The System reports success to the Doctor. The Patient can now view the Medical Record and Invoice on the app.

Branching / Alternative flows:	Sub 1. Invalid Booking Status (At Step 2): 1. The appointment does not exist, or its current status is not CONFIRMED (it might have been canceled or the patient failed to show up). 2. The System rolls back and returns a 404 Not Found or an error stating the appointment cannot be updated. Sub 2. Consultation Time Has Not Arrived (At Step 3): 1. The doctor clicks complete when the actual time is earlier than the scheduled start time. 2. The System intercepts and returns a 400 Bad Request indicating the appointment status cannot be updated at this moment.
Exceptional / Special flows:	Sub 3. Medical Record Creation Failure (At Step 5): 1. The Medical Record Service throws an error. 2. Immediately aborts the local Database Transaction (for the Booking table) and throws an Exception to the user. 3. No garbage data is generated in other services. Sub 4. Invoice Creation Failure (At Step 7): 1. Caused by the Payment Service crashing or throwing an error. 2. Since the Medical Record Service has already successfully created the Medical Record at Step 5, the Schedule Service is forced to automatically trigger a compensating gRPC request (deleteRecord) sent back to the Medical Record Service to delete (revoke) the recently created medical record. 3. Subsequently reports the error to the screen. Sub 5. Appointment Update Failure (At Step 8): 1. The Schedule Service database encounters an issue and fails to update the Booking status. 2. Since both the Medical Record and Invoice were successfully initialized at Steps 5 and 7 respectively, the final catch block simultaneously invokes two gRPC requests, deleteInvoice and deleteRecord, to purge all traces of the newly generated invoice and medical record. 3. Reports a 500 Internal Server Error. Data integrity is strictly and absolutely protected.

Table 1. 4. Examine use case specification

2.2.5. Join remote consultation

Use case name: Join remote consultation	ID: UC-05
Actors: Doctor, Patient, ZegoCloud, Google	Priority: High
	Complexity: Medium
Participants and concerns:	

- Doctor and Patient: Desire a smooth, secure, and punctual video and audio connection experience - ZegoCloud: Provides Video Streaming infrastructure. Requires a unique callId (Room ID) to bridge two users into a single session - System: Ensures only the designated Doctor and Patient of that specific appointment can enter the room while controlling the call room's duration	
Brief description: An actor (Doctor or Patient) interacts with the telemedicine appointment on the Mobile App. The system grants a unique Room ID for that appointment after verifying the right person and right time (permitted to enter up to 5 minutes early). When one user enters the room, the system immediately awakens the other party's device via a Push Notification to invite them to join. The Mobile App then uses this Room ID to establish the video connection through the ZegoCloud SDK suite.	
Relationships: - Association: Doctor, Patient, ZegoCloud, Google - Extend: None - Include: None	
Normal flow of events:	1. The actor clicks the "Join Online Consultation" button on the appointment detail screen of the Mobile App. 2. The application sends an API GET /bookings/video-call/:bookingId request to the system. 3. The system performs Access Guard verification: Queries to find the appointment and checks whether the userId sending the request matches the patientId or doctorId of the appointment. 4. The system validates the time in milliseconds (Threshold rules): - The current time must be earlier than the appointment's end time - The current time cannot be earlier than the appointment's start time by more than 5 minutes (300.000 ms) (allowing early entry for preparation) 5. The system generates a secure static-formatted Room ID (Call ID): Room_{bookingId}_{doctorId}_{patientId}. 6. The system routes the Push Notification dispatch: - If the triggering actor is a Patient: Emits an event to trigger the system to send an awakening notification to the Doctor's phone - If the triggering actor is a Doctor: Emits an event to trigger the notification sent to the Patient's phone 7. The system returns the successful Call ID to the actor's device. 8. The Mobile App integrates the Call ID into the ZegoCloud SDK to initialize the video streaming and audio playback pipeline.

Branching / Alternative flows:	Sub 1. Unauthorized Actor (At Step 3): 1. If another doctor (not assigned to the case) attempts to invoke the API using this appointment code. 2. The system rejects and returns a 404 Not Found or an error stating the appointment is not found or unauthorized. Sub 2. Consultation Expired (At Step 4): 1. The actual time has exceeded the end time of the appointment session. 2. The system blocks access and returns a 400 Bad Request stating the consultation hours have ended. Sub 3. Joining Too Early (At Step 4): 1. The actor tries to enter the room prematurely before the startup window (earlier than 5 minutes prior to the appointment). 2. The system blocks access and returns a 400 Bad Request indicating the session has not started yet, requiring the actor to wait.
Exceptional / Special flows:	Sub 4. Push Notification Delivery Failure (Fault Tolerance): 1. During the process of waking up the other party (At Step 6), if RabbitMQ loses connection or Firebase (FCM) rejects the request, the <code>.catch((e) => { })</code> block silently absorbs the error. 2. This means the initiator (Doctor/Patient) still receives the Call ID and enters the room successfully, even though the other party might not get the alert ring. This is an architectural choice (Fault Tolerance) to prevent breaking video connectivity capabilities if a failure occurs on the Push Notification side. Sub 5. User Denies Camera/Mic Permissions on Mobile OS: 1. Although the backend API successfully returns the Call ID, at Step 8, the client-side ZegoCloud SDK (Mobile) throws an error due to lacking hardware permissions. The Mobile App must catch this exception and display a prompt guiding the user to system Settings to enable permissions.

Table 1. 5. Join remote consultation use case specification

2.2.6. Create schedule change request

Use case name: Create schedule change request	ID: UC-06
Actors: Doctor	Priority: Medium
	Complexity: High
Participants and concerns: - Doctor: Wants the system to easily record their intentions to change their weekly schedule (e.g., switching from morning to afternoon shifts, or adding	

weekend shifts) via the app - Admin: Receives the list of these requests for review, ensuring staffing levels are not depleted before clicking "Approve" - System: Protects data integrity, disallows overlapping working hours, blocks offline shift registrations outside of operating hours, and prevents doctors from spamming multiple simultaneous requests	
Brief description: The doctor sets up a new weekly work schedule on the Mobile App and submits a request to the system. First, the API Gateway validates it to ensure that shifts on the same day do not have overlapping timeframes. Next, the Schedule Service cross-references offline shifts with the clinic's operating hours. If valid and the doctor has no other pending requests, the system creates a Schedule Change Request record with a PENDING status, awaiting review by the Administrator.	
Relationships: - Association: Doctor - Extend: Manage schedule change requests - Include: None	
Normal flow of events:	1. The doctor interacts with the Mobile App: Selects desired shifts for the new week (choosing the day of the week, start time, end time, and Online/Offline examination type) and clicks the "Submit Request" button. 2. The mobile application sends a request to the API Gateway. 3. The API Gateway iterates through the list of shifts to check for overlaps: Ensures that on the same day, no two shifts have intersecting or overlapping timeframes. 4. The API Gateway invokes gRPC to push data down to the Schedule Service. 5. The Schedule Service checks the database to see whether this Doctor already has another schedule change request in a PENDING status. 6. The system opens a Database Transaction and retrieves the clinic's OpeningTime schedule with a read lock (pessimistic_read). 7. The system deeply cross-references each shift submitted by the Doctor: - If the shift is ONLINE: Bypasses time checks (Doctors can conduct online consultations freely) - If the shift is OFFLINE: The start and end times of the shift must fall completely within the clinic's operating hours on that day 8. After passing validation, the system creates the parent main record with a PENDING status and saves all child detail records. 9. The system commits the transaction, completing the data persistence. Returns a success notification: "Schedule

	change request created successfully" to the Doctor.
Branching / Alternative flows:	Sub 1. Overlapping Working Times (At Step 3): 1. If it is detected that on the same day, the doctor registers two intersecting shifts (e.g., 08:00 - 11:00 and 10:00 - 12:00), the system fails-fast, immediately blocks it, and returns a 400 Bad Request error: "Working times must not overlap." Sub 2. Pending Request Already Exists (At Step 5): 1. The doctor has a previous PENDING form that the Admin has not yet processed. The system refuses to overwrite it to avoid database clutter, returning a 400 Bad Request error: "You already have a pending request." Sub 3. Invalid Offline Working Hours (At Step 7): 1. The doctor registers an offline consultation on a day the clinic is closed, or the shift hours exceed operating hours. The system rolls back the transaction and returns a 400 Bad Request error: "Date/time is invalid relative to the clinic's operational schedule."
Exceptional / Special flows:	Sub 4. Database Failure During Data Persistence: 1. While executing Step 8, if the database encounters an issue (saving the parent table succeeds but fails at the child table). The system throws an error and catches it in the catch block. Immediately triggers a rollback of the entire Transaction to prevent saving meaningless parent records. Reports a 500 Internal Server Error externally, maintaining system integrity.

Table 1. 6. Create schedule change request use case specification

2.2.7. Handle schedule change requests

Use case name: Handle schedule change request	ID: UC-07
Actors: Admin	Priority: High
	Complexity: Medium
Participants and concerns: - Admin: Reviews the request form, cross-references it with the clinic's staffing situation to make an Approved or Rejected decision (with a reason) - Doctor: As the beneficiary, wants to receive the final result regarding whether their proposed schedule change request has been accepted by the clinic - System: Must ensure that a request cannot be approved twice and must execute the schedule template replacement process entirely without leaving behind data garbage	
Brief description: The Admin views a list of pending schedule change requests (PENDING) on the CMS Web interface. The Admin has the authority to Approve (APPROVED) or Reject (REJECTED). If Rejection is selected, the request is closed,	

and the system retains the doctor's current working schedule. If Approval is selected, the system performs an "overhaul": Deletes the entire old working schedule configuration (DoctorWeeklyTemplate) of that doctor, and then copies all new time slots from the request form to formally apply them.	
Relationships: - Association: Admin - Extend: Manage schedule change requests - Include: None	
Normal flow of events:	1. The Admin views a request form on the CMS Web and clicks "Approve". 2. The web sends an API call to the system carrying the ID of the request form. 3. The system opens a Database Transaction to initiate the sequence of operations. 4. The system queries to find the ScheduleChangeRequest record by its ID. Its current status must strictly be PENDING. This record is locked by the system using pessimistic_write (Write Lock - Exclusive Lock). 5. The system updates the status of this Request record to APPROVED. 6. The system executes a Delete command: Wipes out all records belonging to this doctor in the Weekly Template Configuration table (DoctorWeeklyTemplate). 7. The system proceeds with an Overwrite (Insert): Scans the detail array (ScheduleChangeRequestDetails) attached to the request, transforms them into brand-new DoctorWeeklyTemplate records, and saves them directly into the Database. 8. The system commits the transaction, releasing the exclusive lock. 9. Returns a success notification: "Schedule change request updated successfully" to the Admin. The doctor officially applies the new work schedule starting from the following week.
Branching / Alternative flows:	Sub 1. Rejection Scenario (REJECTED): 1. At Step 1, the Admin selects "Reject", and the system requires the Admin to enter an additional rejection reason (rejectedReason). 2. At Step 5, the system changes the Request status to REJECTED and saves the reason into the Database. 3. The system entirely bypasses Steps 6 and 7 (no impact on the DoctorWeeklyTemplate table). Proceeds straight to committing the transaction. The doctor keeps their old work schedule. Sub 2. Request Does Not Exist or Has Already Been Processed (At Step 4):

	1. In the event that two Admins are on the system simultaneously, Admin A clicks Approve 1 second prior. When Admin B clicks Approve afterwards, the system scans and no longer finds the PENDING status. 2. The system calls a Transaction Rollback and releases the lock. Returns a 404 Not Found error: "Request not found or has already been processed" to prevent Admin B from distorting the data.
Exceptional / Special flows:	Sub 3. System Error or Database Error: 1. If an error occurs during the old schedule deletion operation (Step 6) or the new schedule overwrite (Step 7) – for example, the database server loses network connectivity – the command falls into the catch block. 2. The system rolls back the entire Transaction. 3. This is extremely critical: It ensures that the old schedule is not deleted if the new schedule has not yet been written, preventing the system from falling into a state where "the doctor completely loses their work schedule." The system reports a 500 Internal Server Error externally.

Table 1. 7. Handle schedule change request use case specification

2.2.8. Pay invoices online

Use case name: Pay invoices online	ID: UC-08
Actors: Patient, VNPay	Priority: Very high
	Complexity: Very high
Participants and concerns: - Patient: Wants to pay invoices securely and quickly, with the invoice status on the app updating automatically right after funds are deducted by the bank - VNPay: The intermediary payment gateway. Requires precise parameter transmission, security signatures, and proper standard response codes (RspCode = 00, 97, 04...) returned when Webhooks are invoked - System: Wants automatic debt settlement with absolute accuracy; must not clear debts with incorrect amounts; must not record a single invoice twice	
Brief description: The patient selects an unpaid invoice to settle. The system generates a unique transaction code and redirects the patient to VNPay's card entry page. Following successful bank fund deductions, VNPay's server secretly triggers a Webhook (IPN) back to the Clinic system. The system verifies the security signature, cross-references the amount, updates the invoice status to Paid (PAID), and returns a confirmation response to VNPay.	
Relationships: - Association: Patient - Extend: Manage invoices	

- Include: None Normal flow of events:	1. Payment initiation: - The patient clicks "Pay via VNPay" for an Invoice on the mobile application - The system (Payment Service) opens a Database Transaction and queries the Invoice using a read lock (pessimistic_read). Ensures the invoice exists and is in an unpaid status (UNPAID) - The system generates a unique transaction code txnRef (concatenating Timestamp + Invoice ID + Random String). Runs a continuous query loop to guarantee this txnRef code is 100% free of duplicates within the Database - The system drafts and saves a new PaymentTransaction record with a PENDING status - The system sorts outgoing parameters, applies the Hmac-SHA512 algorithm alongside a Secret Key to hash them into a Security Signature (vnp_SecureHash) - The system returns the VNPay URL to the device. The mobile app opens the link for the patient to enter bank card credentials 2. Automated Confirmation: - After the bank deducts the funds, the VNPay server automatically triggers a POST request to the system's Webhook API carrying payload data - The system extracts the signature variable, re-hashes the remaining parameters, and compares the two signatures. Confirms that the data genuinely originates from VNPay rather than a hacker forgery - The system opens a Database Transaction, queries to find the PaymentTransaction record matching the txnRef code, and applies a hard lock via pessimistic_write - The system performs logic validation: The amount reported by VNPay matches 100% with the amount stored in the DB, and the transaction status must be PENDING - Inspects the ResponseCode variable from VNPay. Reporting success (00), the system transitions the Transaction status to SUCCESS while simultaneously updating the Invoice to PAID alongside the payment timestamp - Commits the Transaction and returns an HTTP 200 response alongside RspCode = '00' to the VNPay server (signaling that the system has successfully
---	---

	cleared the debt)
Branching / Alternative flows:	Sub 1. Invalid Invoice: 1. If the invoice has already been settled previously, the system rolls back and refuses to generate a link, returning a 404 Not Found error stating the requested invoice is not found. Sub 2. Hacker Forged Webhook: 1. The Hash signature fails to match. The system immediately rejects DB manipulations, returning RspCode = '97' (Invalid Checksum) to VNPay. Sub 3. Amount Mismatch: 1. The amount reported by VNPay does not match the DB. Returns RspCode = '04' (Invalid Amount) Sub 4. Transaction Already Recorded Previously: 1. Occasionally due to network latency, VNPay sends the Webhook a second time. The system blocks it because the transaction status is no longer PENDING. Returns RspCode = '02' (Order already confirmed). Prevents duplicate fund additions or log entries. Sub 5. Payment Failure: 1. The patient enters an incorrect card password, lacks sufficient funds, or presses cancel. VNPay sends a Webhook carrying an error code (ResponseCode != 00). The system updates the Transaction record to FAILED, storing the error code for future reconciliation. The invoice retains its UNPAID status
Exceptional / Special flows:	Sub 6. Database Access Failure during Webhook Phase: 1. If the database server crashes or overloads when VNPay triggers the IPN. The command falls into the catch block. 2. The system automatically rolls back the transaction (strictly avoiding partial data persistence). 3. Unlike standard APIs that return HTTP 500, for VNPay Webhooks, the system must bypass standard errors and return a defined error code so VNPay understands the clinic system is undergoing maintenance, prompting VNPay to automatically retry invoking that Webhook later.

Table 1. 8. Pay invoices online use case specification

2.2.9. View invoices

Use case name: View invoices	ID: UC-09
Actors: Nurse, Admin	Priority: High
	Complexity: Low

Participants and concerns: - Admin: Concerned with a panoramic view: how many invoices have been collected, how many are unpaid (UNPAID), and tracking clinic revenue - Nurse: Concerned with immediacy: being able to easily type a patient's ID to instantly retrieve the invoice from their just-concluded examination in order to quote the cash payment amount - System: Must strictly differentiate permissions: if a Patient calls this API, they are only allowed to view their own invoices; but if an Admin / Nurse calls it, they are granted permission to view all invoices across the clinic	
Brief description: The Administrator or Nurse logs into the CMS Web system and opens the Invoice Management page. The system loads a tabular list of all invoices in the clinic (sorted with the newest at the top). The actor can enter filter conditions such as invoice status or patient ID to quickly extract the required data.	
Relationships: - Association: Admin, Nurse - Extend: Manage invoices - Include: None	
Normal flow of events:	1. The actor (Admin or Nurse) opens the "Invoice Management" page on the CMS Web dashboard. 2. The browser sends a GET /invoices request (accompanied by default pagination parameters page=1, limit=10) to the API Gateway system. 3. The system verifies the Token and Permissions (AccessGuard). Detects that the role invoking the API is either Admin (1) or Nurse (4). 4. The system bypasses the hard-coded user ID constraint (if it were a Patient – Role 3, the system would automatically lock down the transmitted patientId to their exact own ID to prevent snooping on others' invoices). 5. The API Gateway invokes gRPC to push filter parameters down to the Payment Service. 6. The Payment Service initializes a QueryBuilder in the Database: - Sorts data by default from newest to oldest (ORDER BY createdAt DESC) - Fetches the precise row count based on the Skip / Take formula (Pagination) 7. The system returns the resulting data array alongside pagination information (total, page, limit) back to the API Gateway. 8. The Web interface renders the table list of Invoices onto the screen.
Branching / Alternative flows:	Sub 1. Flipping to a Page with No Data: 1. The actor clicks to navigate to an excessively high page number (e.g., Page 100), exceeding the total actual invoice

	count present in the Database. 2. The QueryBuilder still executes normally but fetches 0 rows. Returns a result as an empty data array data: []. The Web interface does not throw an error and simply displays "No data available.".
Exceptional / Special flows:	Sub 2. Loss of Access Rights (Unauthorized): 1. The Admin / Nurse's login session token expires, or a user outside the three permitted groups (Admin / Nurse / Patient) deliberately attempts to invoke this API. The system blocks it right at the outermost perimeter and returns a 401 Unauthorized or 403 Forbidden error. Sub 3. Database Server Interruption: 1. Trapped inside the catch block, logs the incident and returns a 500 Internal Server Error, displaying a "System error" notification on the Web interface.

Table 1. 9. View invoices use case specification

2.2.10. Confirm cash payment

Use case name: Confirm cash payment	ID: UC-10
Actors: Nurse	Priority: High
	Complexity: Medium
Participants and concerns: - Nurse: Wants to operate extremely swiftly on the CMS Web to clear debts for patients standing at the counter right after counting sufficient cash - Patient: Wants their mobile application to instantly update the invoice status to "Paid" right upon handing cash to the nurse for peace of mind when leaving - System: Needs to accurately log the audit trail of which specific nurse (cashierId) collected this sum to easily reconcile funds at the end of the day. Strictly blocks double-payment scenarios (where a patient has just completed a VNPay payment while the nurse coincidentally triggers cash collection)	
Brief description: The patient pays cash directly at the counter. The nurse finds the patient's invoice on the CMS Web system and clicks the "Cash Paid" button. The system locks the invoice record and checks whether it is still unpaid. If valid, the system transitions the invoice to Paid status (PAID), records the payment method as Cash (CASH), registers the collection timestamp, and notably stores the ID of that Nurse for accountability (Audit).	
Relationships: - Association: Admin, Nurse - Extend: Manage invoices - Include: None	
Normal flow of	1. The patient hands over cash to the Nurse at the counter. 2. The nurse finds the patient's invoice on the CMS Web and

events:	clicks the "Cash Payment" button. 3. The browser sends an API request (PATCH /invoices/cash-paid/:id) to the system. 4. The API Gateway authenticates the Token (AccessGuard). Validates permissions to ensure this is a Nurse (Role 4) and extracts this Nurse's ID (cashierId). 5. The API Gateway invokes gRPC to push data down to the Payment Service. 6. The Payment Service opens a Database Transaction. 7. The Payment Service queries to find the Invoice by ID with a mandatory condition that its status is UNPAID. Simultaneously, the system hard-locks this record using a write lock (pessimistic_write). 8. Following a successful lock, the system proceeds to update data fields: - status = PAID - paymentMethod = CASH - cashierId = [Nurse's ID] - paidAt = [Current Timestamp] 9. The system commits the transaction, permanently applying changes and releasing the database lock. 10. Returns a success notification: "Cash payment confirmed successfully." (At this point, the patient refreshing the Mobile App will see the invoice turn green).
Branching / Alternative flows:	Sub 1. Preventing Double-Payment / Invalid Invoice (At Step 7): 1. Hypothesis: A patient standing at the counter successfully clicks VNPay payment on their phone. Simultaneously, the nurse clicks the "Cash Collection" button on the computer. 2. Thanks to the pessimistic_write lock mechanism within the Transaction, if the VNPay Webhook invokes 1 millisecond earlier, it transitions the invoice status to PAID. 3. The Nurse's flow executing immediately afterward queries the Database with the condition status = UNPAID. The system immediately returns empty (no satisfying record found). 4. The system automatically rolls back and returns a 404 Not Found error. The nurse will realize the invoice has been settled through another method and avoids erroneous overwriting.
Exceptional / Special flows:	Sub 2. Database Incident (System Error): 1. If the server database encounters a connection drop right in the middle of updating (Step 8). The system falls into the catch block, and the entire transaction is rolled back to the original UNPAID state to ensure the nurse can retry operations once the network stabilizes, preventing

	corrupted data generation. The system returns a 500 Internal Server Error.
--	--

Table 1. 10. Confirm cash payment use case specification

3. Non-functional requirements

3.1. Performance Requirements

- Under standard network conditions (minimum 10 Mbps connection), the system must guarantee a maximum API response time of under 3.0 seconds for general transactional requests. High-priority actions, such as fetching real-time queue states or active working shifts, should return data within 1.5 seconds
- The containerized NestJS microservices backend must be optimized to handle concurrent requests efficiently. By utilizing RabbitMQ as an asynchronous message broker, high-overhead operations – such as sending push notifications via FCM – must be decoupled from the main request-response cycle, preventing thread blocking and maintaining high throughput during peak hours
- Frequently accessed static or semi-static data, such as clinical department catalogs and active token whitelists, must be cached using an in-memory Redis database. This caching strategy must reduce redundant read queries to the MySQL database, optimizing data retrieval and minimizing database connection overhead

3.2. Safety requirements

- Because core clinical operations – such as appointment scheduling, post-consultation payment processing, and electronic health record (EHR) management – are decoupled into isolated microservices with their own dedicated database instances, traditional local database transactions cannot guarantee data integrity across these distributed boundaries. To ensure eventual consistency and robust data integrity, the system implements an Orchestrator-based Saga Pattern. Within this distributed workflow, the Schedule Service acts as the central orchestrator (coordinator). It sequences and initiates local transactions across participating services, such as requesting invoice generations in the Payment Service and preparing clinical records in the Medical Record Service. If any individual step fails during this distributed chain – such as a VNPay integration failure or an EHR database write error – the Schedule Service immediately intercepts the failure and dispatches a sequence of compensating transactions (such as voiding the generated invoice, freeing up the blocked doctor's shift, or deleting temporary diagnostic records) to revert all databases back to a pristine, consistent state and prevent orphaned data
- While cross-service consistency is managed via distributed Saga workflows, each independent microservice must still enforce strict ACID compliance

within its own local boundary. Any transactional database operations executed within a single service – such as updating an invoice status from "Unpaid" to "Paid" in the MySQL database or inserting a list of prescribed medications – must be wrapped in local database transactions to prevent partial writes and guarantee absolute local consistency

- To prevent physical system crashes and local operational conflicts, all core infrastructural dependencies (MySQL, Redis, RabbitMQ) must be containerized using Docker. This ensures that any runtime error or memory leak in a single service remains isolated and does not compromise the host system's stability or other running processes
- To facilitate seamless end-to-end monitoring, rapid debugging, and comprehensive audit trails across the decentralized microservices architecture, every incoming transaction must be uniquely identifiable. Upon receiving any external request, the API Gateway must dynamically generate and assign a unique, immutable correlationId created by concatenating the requester's client IP address with a globally unique identifier (UUIDv4). This correlationId must be automatically injected into request headers and propagated downstream to all participating microservices through synchronous gRPC metadata and asynchronous RabbitMQ message envelopes. By embedding this identifier across service boundaries, the Winston logging library can systematically tag all distributed logs under a single transactional thread, allowing administrators to trace the complete lifecycle of any request and isolate failures instantly.

3.3. Security requirements

- The system must leverage JWT as the primary standard for secure, cross-platform session management. Upon successful login, the authentication service issues a short-lived access token and a long-lived refresh token. To allow immediate session revocation, token states must be validated against a Redis-backed whitelist
- Token storage strategies must be strictly tailored to the specific client runtime environment. The ReactJS-based web administration portal must store tokens in secure, HttpOnly, and SameSite cookies to mitigate XSS risks. Concurrently, the Flutter mobile application must store tokens using encrypted local storage via the Flutter Secure Storage package
- All patient demographic profiles, credentials, and sensitive electronic health records must be protected both in transit and at rest. Passwords must be hashed using bcrypt before database insertion. All external network transmissions between client applications and the API Gateway must be strictly encrypted over HTTPS
- To safeguard the distributed microservices infrastructure against brute-force attacks, resource exhaustion, and DoS attempts, a strict rate-limiting mechanism must be enforced at the API Gateway level. This mechanism is implemented

utilizing the official NestJS Throttler library, which dynamically monitors and intercepts incoming HTTP traffic based on the client's network origin. The rate limiter enforces a strict system-wide threshold of 60 requests per minute per individual IP address. Any client exceeding this calculated traffic limit must be instantly blocked and returned an HTTP 429 (Too Many Requests) exception, effectively protecting downstream microservices from spam, malicious API scraping, and automated script abuse

3.4. External interface requirements

3.4.1. User interface (UI) requirements

- The web administration portal developed in ReactJS must be highly responsive, support modern web browsers (Google Chrome, Microsoft Edge, Safari), and offer an intuitive, user-friendly layout for clinic nurses to easily manage transactions and drug dispensations
- The Flutter mobile application must adapt seamlessly to various Android screen sizes and resolutions, offering consistent typography, clear diagnostic dashboards, and a specialized interface that displays a prominent join button for telehealth video conferences

3.4.2. Hardware interface requirements

- Hosting Server (Development & Local Deployment): Minimum processor of 2.0 GHz (minimum 6 cores), 16 GB of RAM, and 50 GB of available SSD storage to efficiently host containerized Docker services
- Client Mobile Devices: Android operating system version 10.0 or higher, with integrated front-facing camera capabilities for ZegoCloud video calls

3.4.3. Software interface requirements

- The system must integrate the official VNPay Sandbox SDK to receive payment status notifications securely via server-to-server IPN callbacks
- The platform must interface with ZegoCloud Real-Time Interaction SDK to orchestrate peer-to-peer RTI video and audio consulting rooms
- Asynchronous mobile push notifications must be delivered by interfacing with the FCM API

3.4.4. Communication interface requirements

- External client-to-server communications must rely on standard RESTful web services over secure TCP/IP sockets
- High-performance, low-latency internal communications between microservices (e.g., verifying a user session or retrieving profile data) must be conducted synchronously using gRPC
- Asynchronous event dispatches and distributed message queues must communicate over AMQP via RabbitMQ

3.5. Business rules

- RBAC: The system must strictly enforce API gateway-level authorization based on three predefined user roles
 - Patient: Authorized to register, query real-time doctor schedules, request and book clinic appointments, review personal EHRs, track previous consultation histories, and settle invoice fees online via the integrated VNPay gateway
 - Doctor: Authorized to review assigned appointment schedules, continuously retrieve and analyze the comprehensive EHR history of upcoming patients (for both offline and online consultations), initiate low-latency video calling rooms via the ZegoCloud SDK for online sessions, and submit clinical diagnoses, treatment notes, and digital prescriptions into the MySQL database
 - Nurse: Authorized to retrieve and view detailed digital invoices and prescriptions to facilitate accurate drug dispensing and provide follow-up usage instructions, and manually confirm and update payment statuses for physical cash transactions at the clinic's reception
 - Administrator (Admin): Authorized to perform complete CRUD operations on system-wide master catalogs. This includes managing medical classifications (diseases), standardized medication and drug catalogs, physician scheduling rosters, patient demographics, and overall user account activation or deactivation states
- Post-consultation financial workflow:
 - An invoice is automatically generated only when the consulting doctor submits the final diagnosis and prescription into the EHR database
 - For online payments, a new, unique VNPay transaction reference (txn_ref) must be dynamically generated for every individual online payment attempt initiated by the patient
 - For cash payments, the receptionist / nurse must manually change the payment status of the pre-existing invoice from "UNPAID" to "PAID" via the ReactJS web portal before clinical medications can be dispensed to the patient
- Telehealth consultation validity: Real-time telehealth video conference channels powered by ZegoCloud are only accessible to the assigned patient and doctor. The dedicated join video call interface button must only become active on the client applications five (5) minutes prior to the scheduled appointment slot and must automatically expire once the doctor completes and saves the EHR entry

CHAPTER 2: SYSTEM DESIGN AND IMPLEMENTATION

1. System architecture

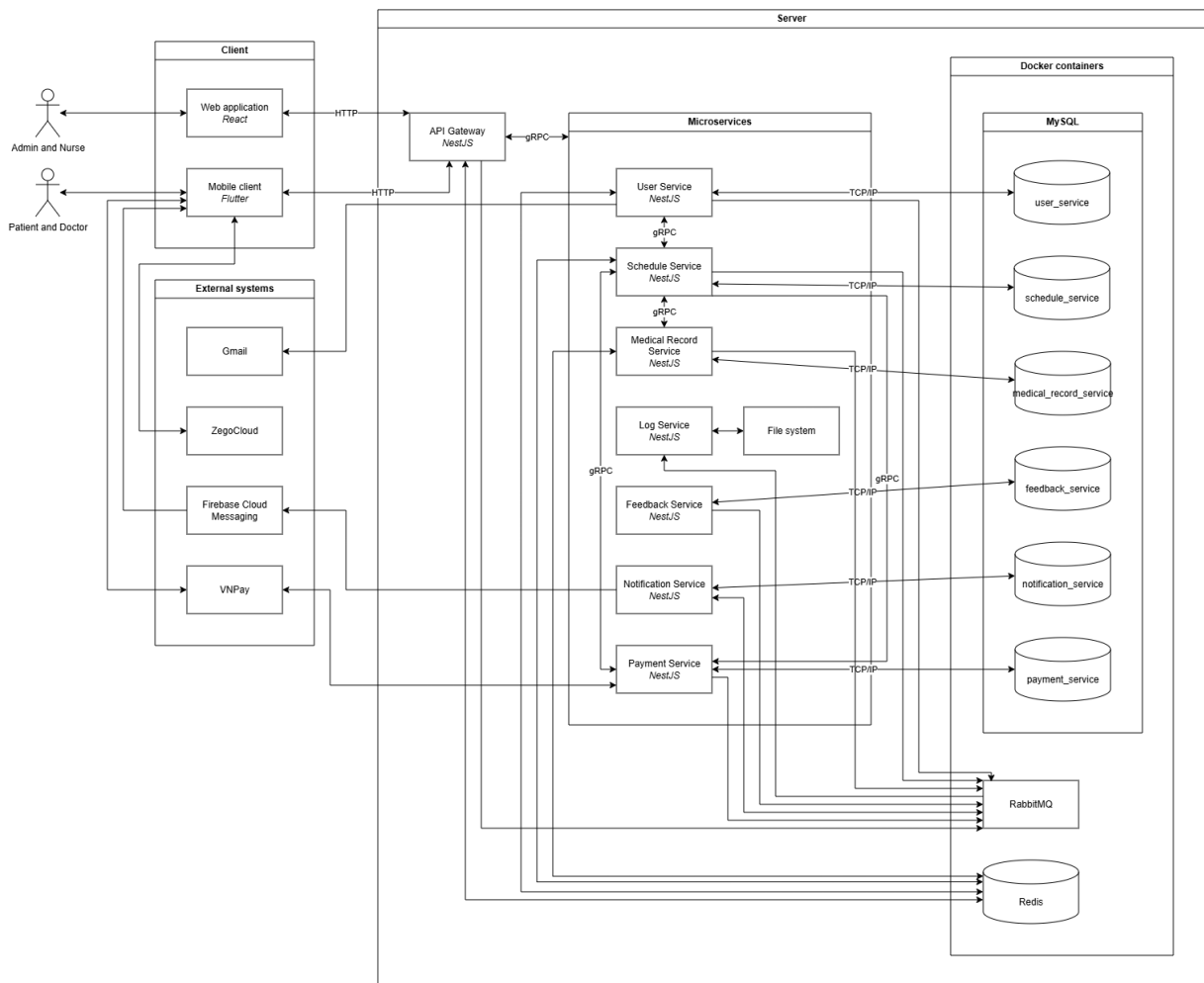


Figure 2. 1. System architecture diagram

To resolve the operational inefficiencies of traditional, monolithic clinic applications, the system is engineered around a containerized, decoupled three-tier microservices architecture. This modern pattern separates the user interface, routing / security gateway, and business logic into independent execution boundaries, ensuring fault isolation, high availability, and horizontal scalability across the entire clinic workflow. The system infrastructure is systematically structured into four core layers:

Client:

The frontend tier is customized to accommodate the distinct environments of different clinical stakeholders:

- **Web application (React):** Serves as the dedicated, responsive terminal for clinic administrators and nurses to coordinate physical operations, confirm cash checkout entries, and manage clinic master catalogs. To mitigate XSS hazards, session tokens are securely handled via **HttpOnly**, **SameSite** cookies

- **Mobile client (Flutter):** Provides a high-performance, native mobile experience for both patients (to coordinate appointment schedules, track medical histories, and complete cashless billing) and doctors (to review patient records and launch telehealth video sessions). Token storage on mobile devices is firmly isolated and encrypted via hardware-backed local secure storage.

API gateway:

Acting as the single entry point for all client requests, the NestJS API Gateway isolates the internal microservices network from public client exposure.

- **Throttling and security:** The Gateway enforces global rate limiting using the native NestJS Throttler library utilizing high-speed, in-memory client IP address tracking, blocking potential DoS and brute-force attempts before they propagate to the backend microservices
- **Gateway caching:** To minimize latency, the gateway implements a local in-memory caching mechanism to store and immediately serve high-traffic, semi-static administrative dashboard metadata, bypassing redundant backend microservice calls
- **Secure webhook routing:** For electronic transactions, the API Gateway acts as the public receiver for asynchronous VNPay Sandbox IPN callbacks, verifying and safely routing the transaction payloads downstream to the internal Payment Service for state synchronization

Decoupled microservices:

The core business logic of the clinic is partitioned into lightweight, autonomous microservices designed around the Database-per-Service paradigm to eliminate single points of failure and database locking bottlenecks:

- **User Service:** Coordinates user registrations, role-based profiles, and access control. It connects directly with the relational database and interfaces with Redis to maintain and validate the stateless session JWT Whitelist
- **Schedule Service:** Acts as the functional "brain" for clinical shifts and Saga transactional workflows. It links directly with Redis to utilize distributed locks, guaranteeing absolute consistency and preventing overbooking conflicts when patients concurrently reserve the same doctor shift
- **Medical Record Service:** Manages EHRs and prescriptions. It leverages Redis cache partitions to accelerate the query performance of heavy patient clinical histories
- **Log Service:** Serves as the centralized diagnostic storage. Rather than performing blocking synchronous logging, other microservices dispatch structured logging payloads asynchronously to RabbitMQ, which are consumed by the Log Service to be structured and written to rolling Winston diagnostic files on the host file system

- **Payment, Notification, and Feedback Services:** Autonomously process cashless gateway integrations, asynchronous FCM pushes, and clinic review metrics respectively

Infrastructure and persistent database:

To ensure complete environmental reproducibility and protect clinical databases from host operating system instability, the entire data persistent layer is containerized using Docker and WSL. Each microservice communicates exclusively with its own dedicated MySQL database instance. Shared caching, token validation, and distributed locks are handled via an isolated Redis container, while asynchronous event propagation is managed by a centralized RabbitMQ container.

Microservices integration protocol:

Communication across the decoupled system boundaries is architected over two complementary network protocols to balance immediate processing requirements with high infrastructure responsiveness:

- **Synchronous Internal Communication (gRPC):** For internal microservice operations requiring real-time, deterministic responses – such as validating an active token session between the gateway and User Service, or looking up doctor rosters within the Schedule Service – the system utilizes gRPC. Operating over binary HTTP/2 connections, gRPC utilizes Protocol Buffers (Proto3) to serialize data payloads into compact binary streams, eliminating the transmission overhead, text-parsing latency, and large payload sizes characteristic of traditional internal REST APIs
- **Asynchronous Event-Driven Messaging (RabbitMQ):** For background workflows that do not require an instantaneous response – such as updating clinical queue statuses, logging system telemetry, sending real-time push notifications via FCM, or executing email OTP transmissions – the system utilizes RabbitMQ as a distributed message broker. Microservices publish message envelopes to designated AMQP exchange queues, where they are buffered and processed asynchronously by the subscribing services. This decoupling effectively shields the system from network spikes, prevents microservice cascading failures, and prevents API thread blocking

2. Data design

2.1. Data model

2.1.1. user_service

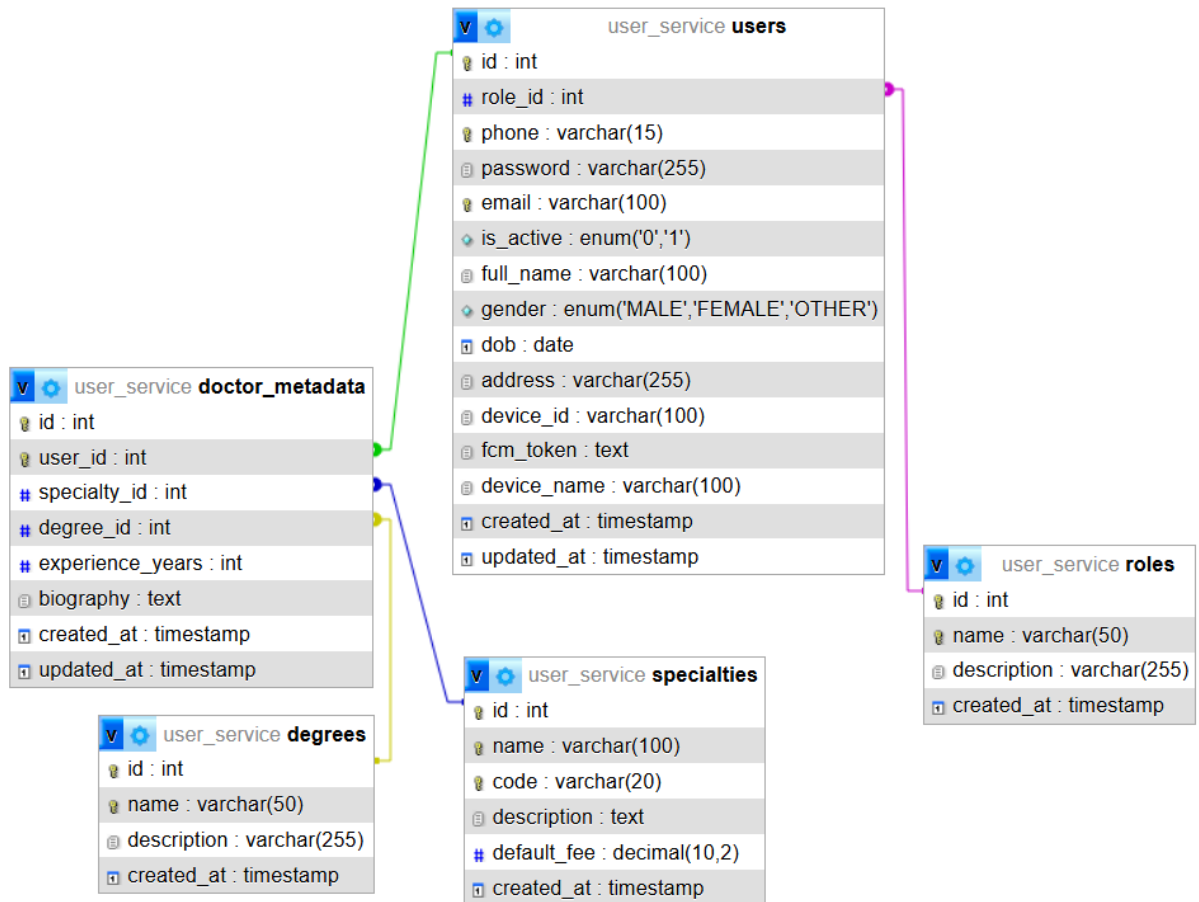


Figure 2. 2. user_service's physical data model

2.1.2. schedule_service

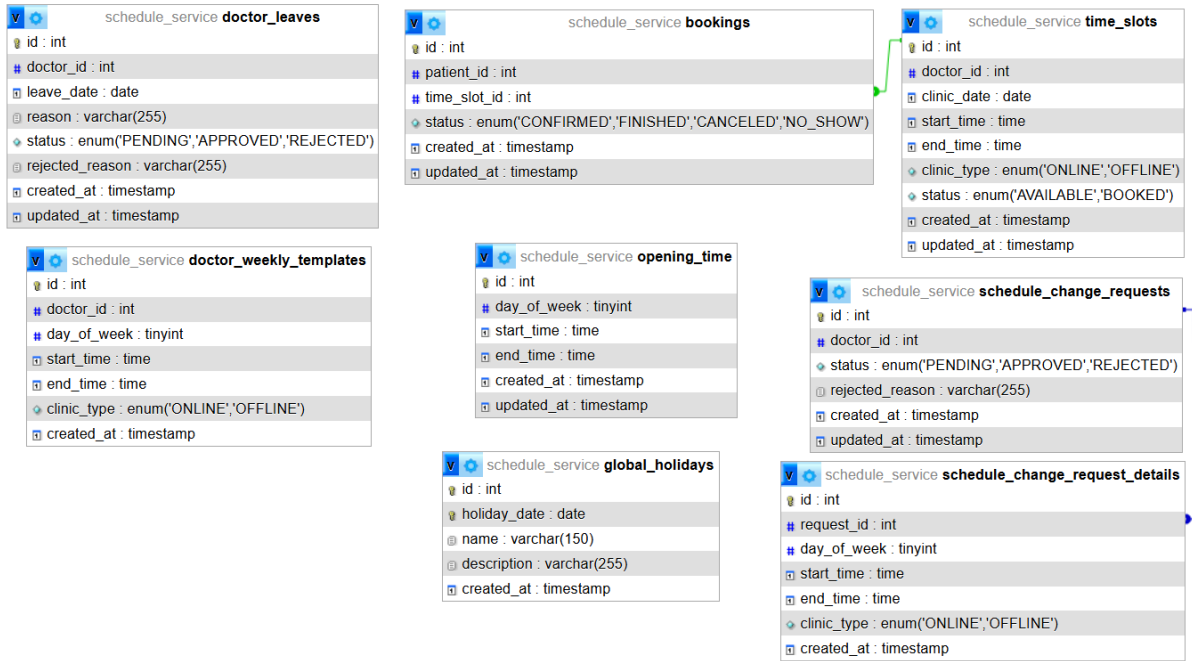


Figure 2.3. *schedule_service's physical data model*

2.1.3. medical_record_service

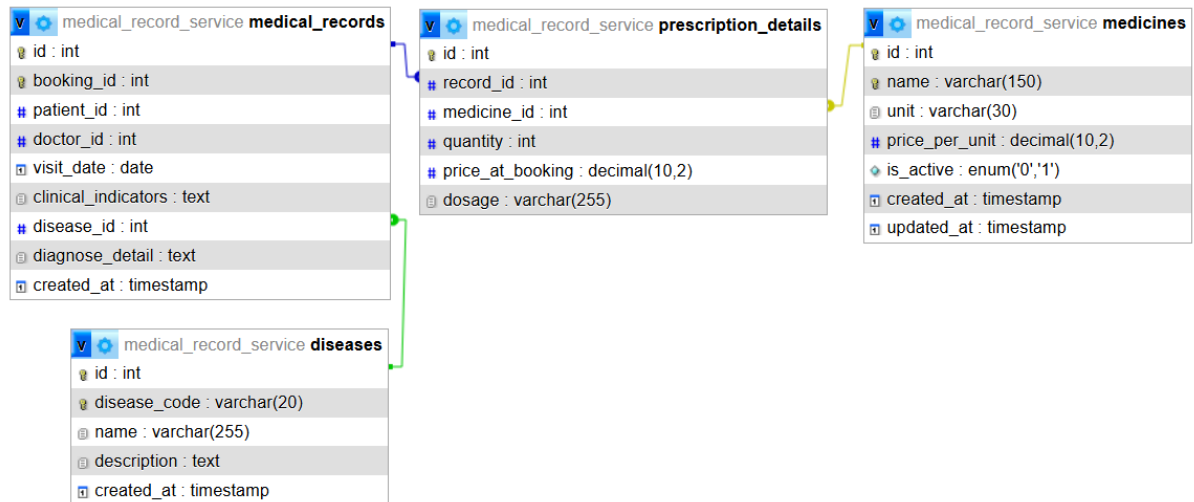


Figure 2.4. *medical_record_service's physical data model*

2.1.4. feedback_service

feedback_service	user_feedbacks
id : int	
# user_id : int	
title : varchar(150)	
content : text	
is_read : enum('0','1')	
created_at : timestamp	
updated_at : timestamp	

Figure 2. 5. feedback_service's physical data model

2.1.5. notification_service

notification_service	notifications
id : int	
# user_id : int	
title : varchar(150)	
content : text	
created_at : timestamp	

Figure 2. 6. notification_service's physical data model

2.1.6. payment_service

payment_service	invoices
id : int	
booking_id : int	
# patient_id : int	
# examination_fee : decimal(10,2)	
# medicine_fee : decimal(10,2)	
# total_amount : decimal(10,2)	
payment_method : enum('CASH','VNPAY')	
status : enum('UNPAID','PAID','CANCELED')	
# cashier_id : int	
paid_at : timestamp	
created_at : timestamp	
updated_at : timestamp	

payment_service	payment_transactions
id : int	
# invoice_id : int	
txn_ref : varchar(100)	
transaction_no : varchar(100)	
# amount : decimal(10,2)	
status : enum('PENDING','SUCCESS','FAILED')	
payment_raw_log : text	
created_at : timestamp	
updated_at : timestamp	

Figure 2. 7. payment_service's physical data model

2.2. Data dictionary

2.2.1. user_service

No.	Field name	Data type	Constraints / Keys	Description
1	id	int	Primary Key, Auto Increment	Unique internal identifier for each role
2	name	varchar(50)	Unique, NOT NULL	Role name (e.g., ADMIN, DOCTOR, PATIENT, NURSE)
3	description	varchar(255)	DEFAULT NULL	Descriptive details regarding the role's scope
4	created_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Date and time when the role record was created

Table 2. 1. roles table

No.	Field name	Data type	Constraints / Keys	Description
1	id	int	Primary Key, Auto Increment	Unique identifier for the medical degree
2	name	varchar(50)	Unique, NOT NULL	Academic degree name (e.g., General Practitioner, Specialist I)
3	description	varchar(255)	DEFAULT NULL	Description of the degree's criteria
4	created_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Creation timestamp of the degree record

Table 2. 2. degrees table

No.	Field name	Data type	Constraints / Keys	Description
1	id	int	Primary Key, Auto Increment	Unique identifier for each medical specialty

2	name	varchar(100)	Unique, NOT NULL	Name of the clinical specialty (e.g., Ophthalmology)
3	code	varchar(20)	Unique, NOT NULL	Short acronym/code representing the specialty
4	description	text	DEFAULT NULL	Detailed description of the department's medical scope
5	default_fee	decimal(10,2)	NOT NULL, DEFAULT 100000.00	Standard base consultation fee for this specialty
6	created_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Creation timestamp of the specialty record

Table 2. 3. specialties table

No.	Field name	Data type	Constraints / Keys	Description
1	id	int	Primary Key, Auto Increment	Unique identifier for the user account
2	role_id	int	Foreign Key (roles.id), NOT NULL	Links the user to an assigned authorization level
3	phone	varchar(15)	Unique, Index, NOT NULL	Phone number used for login and identity verification
4	password	varchar(255)	NOT NULL	Securely hashed user password (using bcrypt)
5	email	varchar(100)	Unique, Index, NOT NULL	User's email address used for OTP verification
6	is_active	enum	DEFAULT '1'	Account status flag (1: Active, 0: Suspended)

7	full_name	varchar(100)	NOT NULL	User's full legal name
8	gender	enum	NOT NULL	Biologically recognized gender of the user
9	dob	date	DEFAULT NULL	Date of birth of the user
10	address	varchar(255)	DEFAULT NULL	Residential physical address of the user
11	device_id	varchar(100)	DEFAULT NULL	Unique physical ID of the mobile client hardware
12	fcm_token	text	DEFAULT NULL	FCM token for mobile push alerts
13	device_name	varchar(100)	DEFAULT NULL	Model / name of the device logging into the system
14	created_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	User account registration timestamp
15	updated_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Timestamp of the most recent user profile update

Table 2. 4. users table

No.	Field name	Data type	Constraints / Keys	Description
1	id	int	Primary Key, Auto Increment	Unique identifier for doctor metadata
2	user_id	int	Foreign Key (users.id), Unique	Map pointer linking back to the parent doctor account
3	specialty_id	int	Foreign Key (specialties.id), NOT NULL	Pointer mapping the doctor to their main specialty
4	degree_id	int	Foreign Key (degrees.id), NOT NULL	Pointer mapping the doctor to

				their medical academic degree
5	experience_years	int	NOT NULL, DEFAULT 0	Active practicing experience measured in years
6	biography	text	DEFAULT NULL	Short medical practicing background biography
7	created_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Timestamp when the metadata record was created
8	updated_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Timestamp of the most recent profile edit

Table 2. 5. doctor_metadata table

2.2.2. schedule_service

No.	Field name	Data type	Constraints / Keys	Description
1	id	int	Primary Key, Auto Increment	Unique identifier of clinic operating schedule
2	day_of_week	tinyint	NOT NULL	Day indicator (0: Sunday, 1-6: Monday to Saturday)
3	start_time	time	NOT NULL	Operating start time of the clinic
4	end_time	time	NOT NULL	Closing time of the clinic
5	created_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Record creation timestamp
6	updated_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Recent calendar update timestamp

Table 2. 6. opening_time table

No.	Field name	Data type	Constraints / Keys	Description
1	id	int	Primary Key, Auto Increment	Unique identifier

				for clinic holidays
2	holiday_date	date	Unique, Index, NOT NULL	Selected calendar date of the holiday
3	name	varchar(150)	NOT NULL	Name of the holiday (e.g., National Day)
4	description	varchar(255)	DEFAULT NULL	Description details of the public holiday
5	created_at	timestamp	NOT NULL. DEFAULT CURRENT_TIMESTAMP	Record creation timestamp

Table 2. 7. global_holidays table

No.	Field name	Data type	Constraints / Keys	Description
1	id	int	Primary Key, Auto Increment	Unique template slot index ID
2	doctor_id	int	NOT NULL, index	References the doctor's user ID
3	day_of_week	tinyint	NOT NULL	Day of week index (0: Sunday, 1-6: Monday to Saturday)
4	start_time	time	NOT NULL	Start time of the recurring shift
5	end_time	time	NOT NULL	End time of the recurring shift
6	clinic_type	enum	NOT NULL	Consultation method (ONLINE: Video Consultation, OFFLINE: Clinic Visit)
7	created_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Roster template configuration date

Table 2. 8. doctor_weekly_templates table

No.	Field name	Data type	Constraints / Keys	Description
1	id	int	Primary Key, Auto Increment	Unique identifier for a generated schedule slot

2	doctor_id	int	NOT NULL, Index	Target doctor's user ID
3	clinic_date	date	NOT NULL, Index	Calendar date of the shift
4	start_time	time	NOT NULL	Start time of this slot
5	end_time	time	NOT NULL	End time of this slot
6	clinic_type	enum	NOT NULL	Target consulting method
7	status	enum	DEFAULT 'AVAILABLE'	Availability status (AVAILABLE: Empty, BOOKED: Reserved)
8	created_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Timestamp when generated by template engine
9	updated_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Timestamp of status modification

Table 2. 9. time_slots table

No.	Field name	Data type	Constraints / Keys	Description
1	id	int	Primary Key, Auto Increment	Unique identifier of the appointment record
2	patient_id	int	NOT NULL, Index	References the patient user ID
3	time_slot_id	int	Foreign Key (time_slots.id), Index	References the selected daily shift
4	status	enum	DEFAULT 'CONFIRMED'	Appointment status (CONFIRMED: Confirmed, FINISHED: Attended, CANCELED: Canceled, NO_SHOW: Absent)
5	created_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Creation timestamp of the booking transaction
6	updated_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Last modification timestamp

Table 2. 10. bookings table

No.	Field name	Data type	Constraints / Keys	Description
1	id	int	Primary Key, Auto Increment	Unique identifier for the leave application
2	doctor_id	int	NOT NULL, Index	References the doctor's user ID
3	leave_date	date	NOT NULL	Requested calendar date of absence
4	reason	varchar(255)	NOT NULL	Documented reason for the leave request
5	status	enum	DEFAULT 'PENDING'	Review status (PENDING: Awaiting decision, APPROVED: Approved, REJECTED: Declined)
6	rejected_reason	varchar(255)	DEFAULT NULL	Administrative feedback why request was rejected
7	created_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Leave application submission timestamp
8	updated_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Roster status updating timestamp

Table 2. 11. doctor_leaves table

No.	Field name	Data type	Constraints / Keys	Description
1	id	int	Primary Key, Auto Increment	Unique identifier for schedule change requests
2	doctor_id	int	NOT NULL	References the doctor user ID
3	status	enum	DEFAULT 'PENDING'	Change approval state

4	rejected_reason	varchar(255)	DEFAULT NULL	Reason for rejecting the schedule swap
5	created_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Submission timestamp
6	updated_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Resolution timestamp

Table 2. 12. schedule_change_requests table

No.	Field name	Data type	Constraints / Keys	Description
1	id	int	Primary Key, Auto Increment	Unique identifier for detail records
2	request_id	int	Foreign Key (schedule_change_requests.id)	Links proposed shifts to swap request parent record
3	day_of_week	tinyint	NOT NULL	Proposed weekday code (0 - 6)
4	start_time	time	NOT NULL	Proposed start time for shift
5	end_time	time	NOT NULL	Proposed closing time for shift
6	clinic_type	enum	NOT NULL	Proposed clinic consulting type
7	created_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Record creation date

Table 2. 13. schedule_change_request_details table

2.2.3. medical_record_service

No.	Field name	Data type	Constraints / Keys	Description
1	id	int	Primary Key, Auto Increment	Unique identifier for disease
2	disease_code	varchar(20)	Unique, Index, NOT NULL	Standardized ICD-10 medical code
3	name	varchar(255)	NOT NULL	Diagnosis title description
4	description	text	DEFAULT NULL	Pathological parameters and

				notes
5	created_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Creation timestamp of the disease catalog

Table 2. 14. diseases table

No.	Field name	Data type	Constraints / Keys	Description
1	id	int	Primary Key, Auto Increment	Unique diagnostic medical record ID
2	booking_id	int	Unique, NOT NULL	Mapping ID linking back to original booking event
3	patient_id	int	NOT NULL, Index	References target patient user ID
4	doctor_id	int	NOT NULL	References consulting doctor's user ID
5	visit_date	date	NOT NULL	Calendar date of consultation
6	clinical_indicators	text	NOT NULL	Patient vital signs (blood pressure, heart rate, temp)
7	disease_id	int	Foreign Key (diseases.id), DEFAULT NULL	Main diagnosis matched to ICD catalog
8	diagnose_detail	text	NOT NULL	Doctor's detailed medical examination notes
9	created_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	EHR catalog creation date

Table 2. 15. medical_records table

No.	Field name	Data type	Constraints / Keys	Description
1	id	int	Primary Key, Auto	Unique identifier for

			Increment	pharmaceutical
2	name	varchar(150)	Unique, Index, NOT NULL	Brand/Generic name of the drug
3	unit	varchar(30)	NOT NULL	Dispensing unit (e.g., Pill, Bottle)
4	price_per_unit	decimal(10,2)	NOT NULL	Price for a single unit of medicine
5	is_active	enum	DEFAULT '1'	Active flag for inventory (1: Active, 0: Suspended)
6	created_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Medicine record configuration timestamp
7	updated_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Latest metadata modification timestamp

Table 2. 16. medicines table

No.	Field name	Data type	Constraints / Keys	Description
1	id	int	Primary Key, Auto Increment	Unique identifier for prescription line item
2	record_id	int	Foreign Key (medical_records.id), NOT NULL	References parent patient diagnostic record (EHR)
3	medicine_id	int	Foreign Key (medicines.id), NOT NULL	References corresponding medication item
4	quantity	int	NOT NULL	Prescribed quantity of units
5	price_at_booking	decimal(10,2)	NOT NULL	Snapshot of unit price at booking to prevent rate fluctuations
6	dosage	varchar(255)	NOT NULL	Instructions for consumption (e.g., times per day)

Table 2. 17. prescription_details table

2.2.4. feedback_service

No.	Field name	Data type	Constraints / Keys	Description
1	id	int	Primary Key, Auto Increment	Unique identifier of system feedback record
2	user_id	int	NOT NULL	Submitting user ID
3	title	varchar(150)	NOT NULL	Brief summary/title of the feedback
4	content	text	NOT NULL	Detailed feedback commentary
5	is_read	enum	NOT NULL, DEFAULT '0'	Review status flag (1: Reviewed, 0: Unread inbox)
6	created_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Submission timestamp
7	updated_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Timestamp when administrator updated reading status

Table 2. 18. user_feedbacks table

2.2.5. notification_service

No.	Field name	Data type	Constraints / Keys	Description
1	id	int	Primary Key, Auto Increment	Unique identifier of notification log
2	user_id	int	NOT NULL, Index	Recipient user ID
3	title	varchar(150)	NOT NULL	Heading title of the notification message
4	content	text	NOT NULL	Body content explaining the event context
5	created_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Notification dispatch timestamp

Table 2. 19. notifications table

2.2.6. payment_service

No.	Field name	Data type	Constraints / Keys	Description
1	id	int	Primary Key, Auto Increment	Unique identifier of system invoice
2	booking_id	int	Unique, NOT NULL	Mapping ID linking back to original booking event
3	patient_id	int	NOT NULL, Index	Target patient user ID
4	examination_fee	decimal(10,2)	NOT NULL	Consulting and clinical visit cost
5	medicine_fee	decimal(10,2)	NOT NULL, DEFAULT 0.00	Calculated total of pharmaceutical prescription costs
6	total_amount	decimal(10,2)	NOT NULL	Combined billing total (examination + medicine)
7	payment_method	enum	DEFAULT NULL	Settling method used by patient
8	status	enum	NOT NULL, DEFAULT 'UNPAID'	Invoice settlement status
9	cashier_id	int	DEFAULT NULL	Nurse user ID processing physical cash
10	paid_at	timestamp	DEFAULT NULL	Date and time when invoice payment was settled
11	created_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Timestamp when invoice was generated
12	updated_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Invoice status update log

				timestamp
--	--	--	--	-----------

Table 2. 20. invoices table

No.	Field name	Data type	Constraints / Keys	Description
1	id	int	Primary Key, Auto Increment	Unique payment transaction ID
2	invoice_id	int	Foreign Key (invoices.id), NOT NULL	Pointer to invoice settled by this transaction
3	txn_ref	varchar(100)	Unique, Index, NOT NULL	System internal tracking code sent to VNPay
4	transaction_no	varchar(100)	DEFAULT NULL	VNPay official payment code response
5	amount	decimal(10,2)	NOT NULL	Transacted monetary sum
6	status	enum	DEFAULT 'PENDING'	Transaction status returned from payment gateway
7	payment_raw_log	text	DEFAULT NULL	Complete raw callback response logs from VNPay IPN
8	created_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Creation timestamp
9	updated_at	timestamp	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Transaction resolution timestamp

Table 2. 21. payment_transactions table

3. Detailed functional design

In this section, the core functions of the application will be presented in detail.

3.1. Book appointment

Purpose: Empowers authenticated patients to search, select, and safely reserve clinical or telehealth consultation shifts with specific doctors while avoiding scheduling conflicts through database-level locking.

User interface:



Figure 2. 8. Book appointment user interface

User interface components:

No.	Control type	Content	Description
1	Button	Back	Return to the previous screen
2	List of buttons	Specialty name	Select specialty
3	Select	Clinic type	Select clinic type
4	Date picker	Clinic date	Select clinic date
5	List of buttons	List of doctor names	Select doctor
6	List of buttons	List of time ranges	Select clinic time

7	Button	Confirm	Confirm
---	--------	---------	---------

Table 2. 22. Book appointment user interface components

Used data:

No.	Table name	Operation			
		Create	Update	Delete	Read
1	specialties				X
2	doctor_metadata				X
3	users				X
4	time_slots		X		X
5	invoices				X
6	bookings	X			X
7	notifications	X			

Table 2. 23. Book appointment used data

Process:

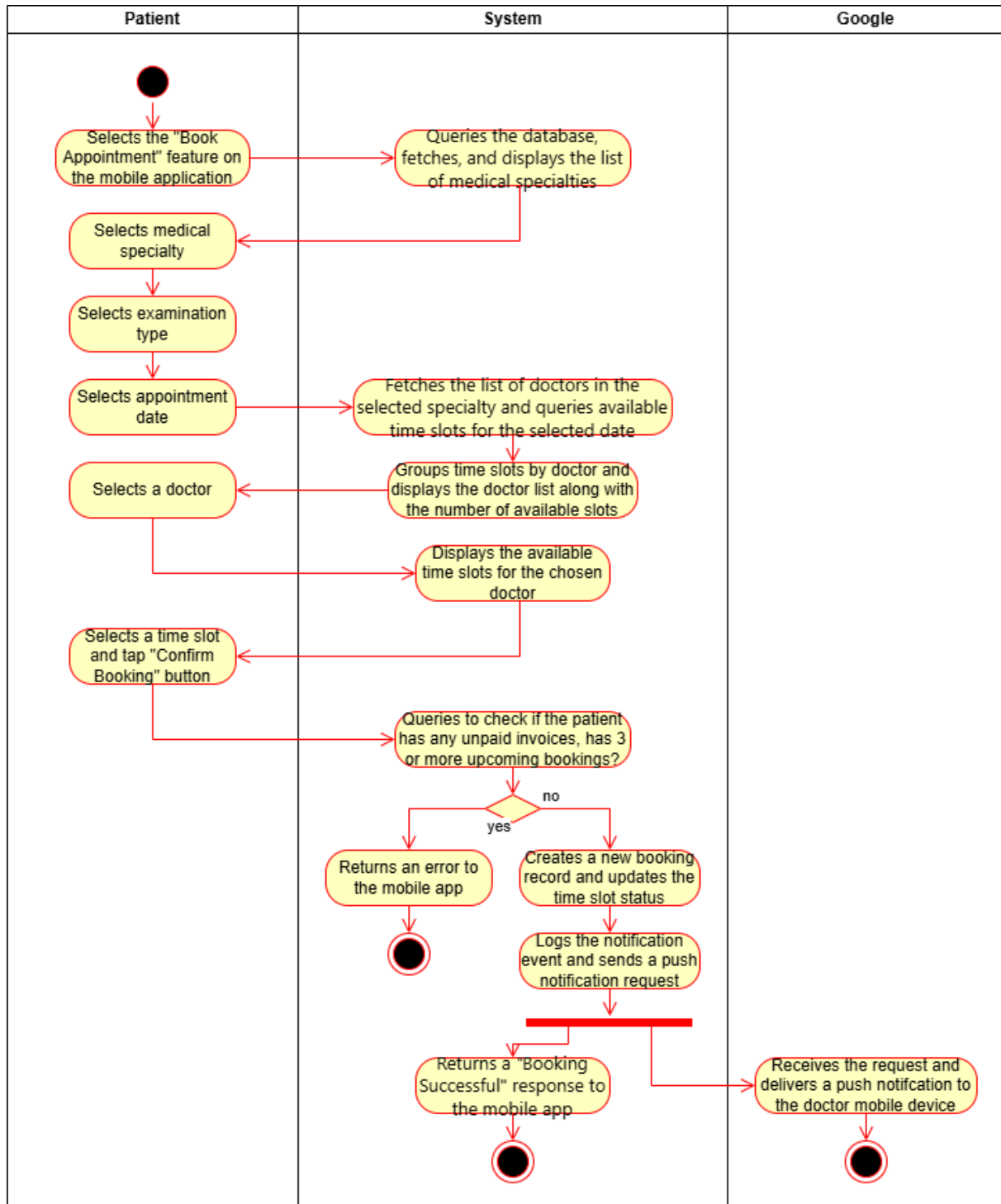


Figure 2. 9. Book appointment process

3.2. Schedule

Purpose: Automates the weekly generation of concrete daily doctor shifts (time slots) on the server, mapping recurring rosters while dynamically skipping clinic holidays and physicians' approved leave dates.

User interface: This function does not have user interface.

Used data:

No.	Table name	Operation			
		Create	Update	Delete	Read
1	time_slots	X			X
2	global_holidays				X
3	doctor_weekly_templates				X
4	doctor_leaves				X

Table 2. 24. Schedule used data

Process:

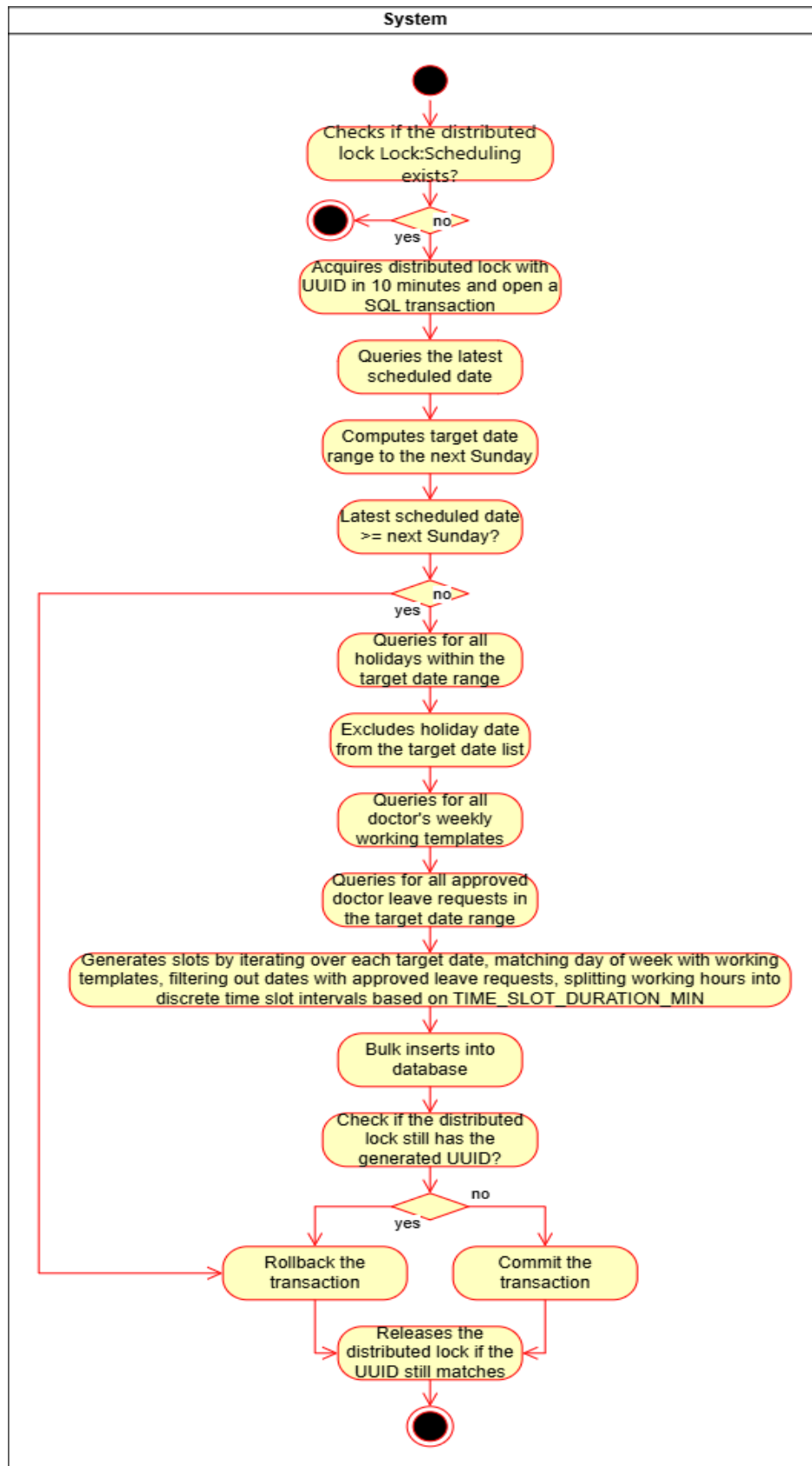


Figure 2. 10. Schedule process

3.3. Clear unused schedule

Purpose: To systematically optimize database storage and maintain clinical schedule hygiene by automatically removing unbooked shifts and canceling expired appointments. This automated, server-side background process periodically executes to delete obsolete, unbooked doctor shifts (time_slots with AVAILABLE status) that have passed without any patient reservations. Concurrently, it identifies clinical appointments (bookings) whose scheduled time frames have already expired, programmatically updating their status to CANCELED to ensure accurate operational metrics and prevent outdated pending records from cluttering the system.

User interface: This function does not have user interface.

Used data:

No.	Table name	Operation			
		Create	Update	Delete	Read
1	time_slots			X	
2	bookings		X		

Table 2. 25. Clear unused schedule used data

Process:

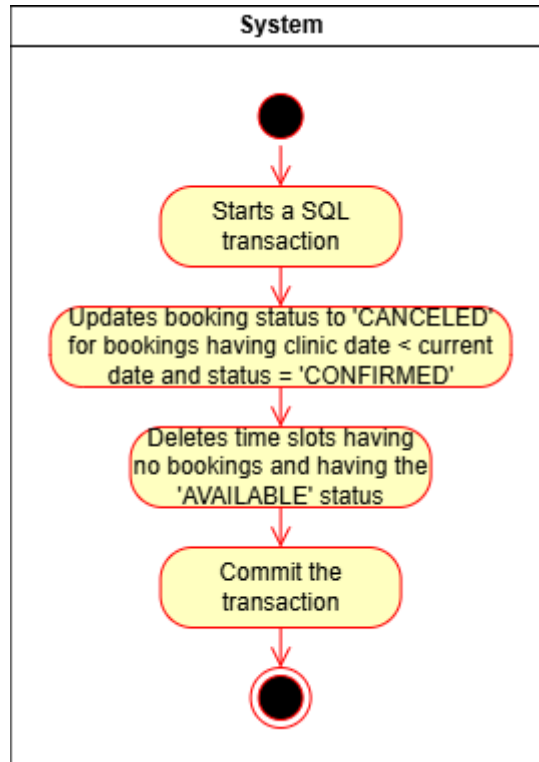


Figure 2. 11. Clear unused schedule process

3.4. Examine

Purpose: Enables doctors to document vital clinical parameters, log standardized diagnostic classifications, and build digital prescriptions during patient consultations, automatically triggering downstream billing.

User interface:

The screenshot shows a mobile application interface for a medical consultation. At the top, there is a back arrow and the title 'Khám bệnh'. Below this is a section titled 'Thông tin lâm sàng' (Clinical information) containing two text input fields: 'Triệu chứng (*)' (Symptoms) and 'Chẩn đoán (*)' (Diagnosis). Below these is a section titled 'Bệnh lý (ICD-10)' (Disease (ICD-10)) which includes a button to remove a selected disease ('H10 - Viêm kết mạc' with a close icon) and a search bar with the text 'H10 - Viêm kết mạc'. Below the search bar is a section titled 'Đơn thuốc' (Prescription) with a '+ Thêm thuốc' (Add medicine) button. This section contains a list of medicines, with 'Paracetamol 500mg (Viên)' selected. Below the medicine list are fields for 'SL' (Quantity), 'Liều dùng (VD: 2 viên/n...)' (Dosage), and 'Ghi chú (Tùy chọn)' (Remarks). At the bottom is a large green button labeled 'Hoàn tất khám bệnh' (Finish examination).

1 ← Khám bệnh

Thông tin lâm sàng

Triệu chứng (*)

Chẩn đoán (*)

Bệnh lý (ICD-10)

H10 - Viêm kết mạc

Tim bệnh lý...

H10 - Viêm kết mạc

Đơn thuốc + Thêm thuốc

Chọn thuốc

Paracetamol 500mg (Viên)

SL Liều dùng (VD: 2 viên/n...

Ghi chú (Tùy chọn)

Hoàn tất khám bệnh

Figure 2. 12. Examine user interface

User interface components:

No.	Control type	Content	Description
1	Button	Back	Return to the previous screen
2	Textbox	Symptoms	Symptoms
3	Textbox	Diagnosis	Diagnosis
4	Button	Disease	Remove disease

5	Textbox	Disease	Select disease
6	Button	Add medicine	Add medicine
7	Textbox	Medicine	Select medicine
8	Button	Remove medicine	Remove medicine
9	Textbox	Quantity	Medicine quantity
10	Textbox	Dosage	Dosage instruction
11	Textbox	Notes	Other usage instructions
12	Button	Confirm	Confirm medical record

Table 2. 26. Examine user interface components

Used data:

No.	Table name	Operation			
		Create	Update	Delete	Read
1	time_slots				X
2	bookings		X		X
3	doctor_metadata				X
4	specialties				X
5	medical_records	X			X
6	medicines				X
7	prescription_details	X			
8	invoices	X			X

Table 2. 27. Examine used data

Process:

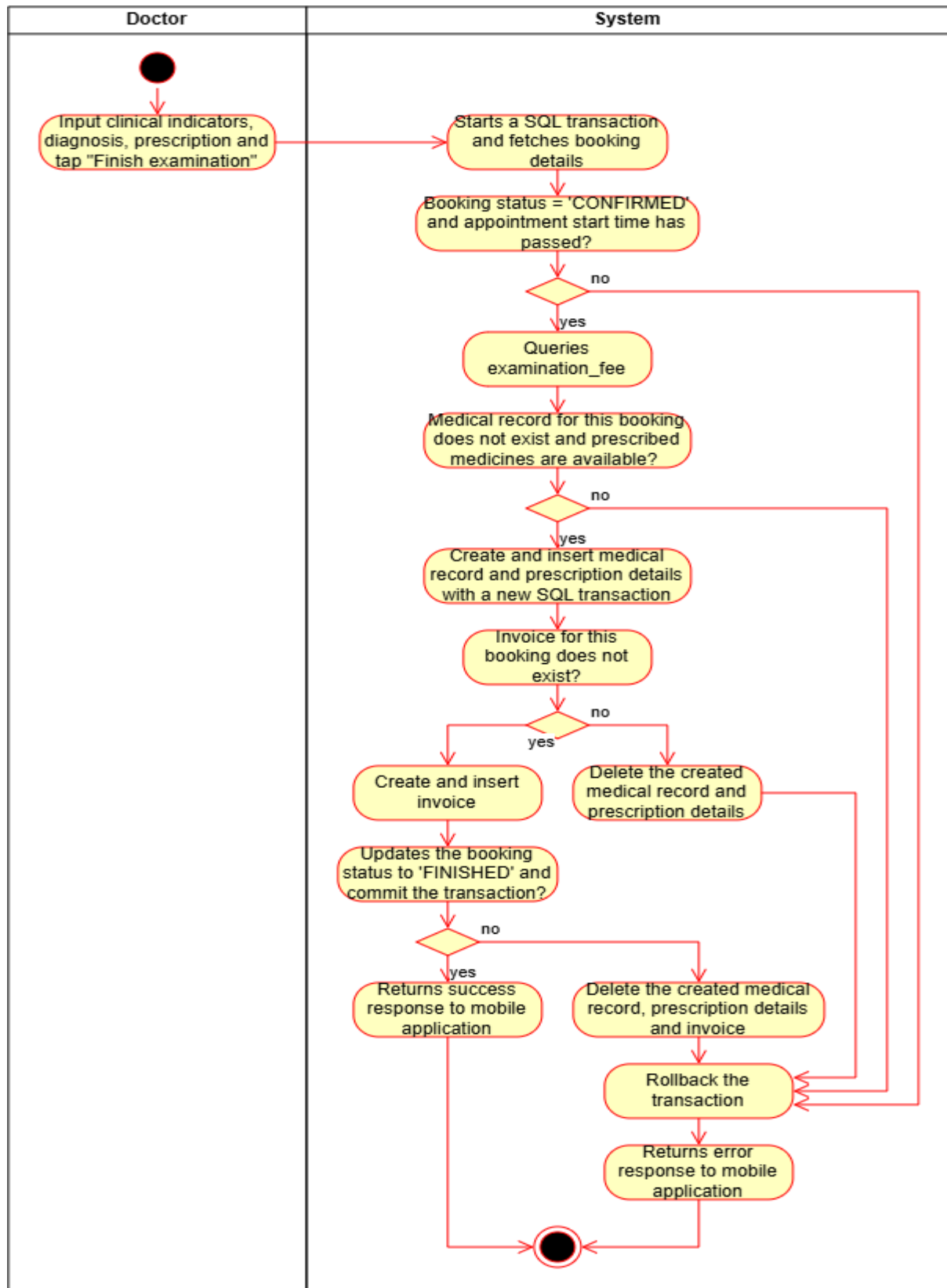


Figure 2. 13. Examine process

3.5. Join remote consultation

Purpose: Establishes an ultra-low-latency RTI virtual video-audio calling room to facilitate telemedicine consultations between doctors and patients at their booked appointment shifts.

User interface:



Figure 2. 14. Join remote consultation user interface

User interface components:

No.	Control type	Content	Description
1	Button	Start video call	Start video call

Table 2. 28. Join remote consultation user interface components

Used data:

No.	Table name	Operation			
		Create	Update	Delete	Read
1	time_slots				X
2	bookings				X
3	users				X
4	notifications	X			

Table 2. 29. Join remote consultation used data

Process:

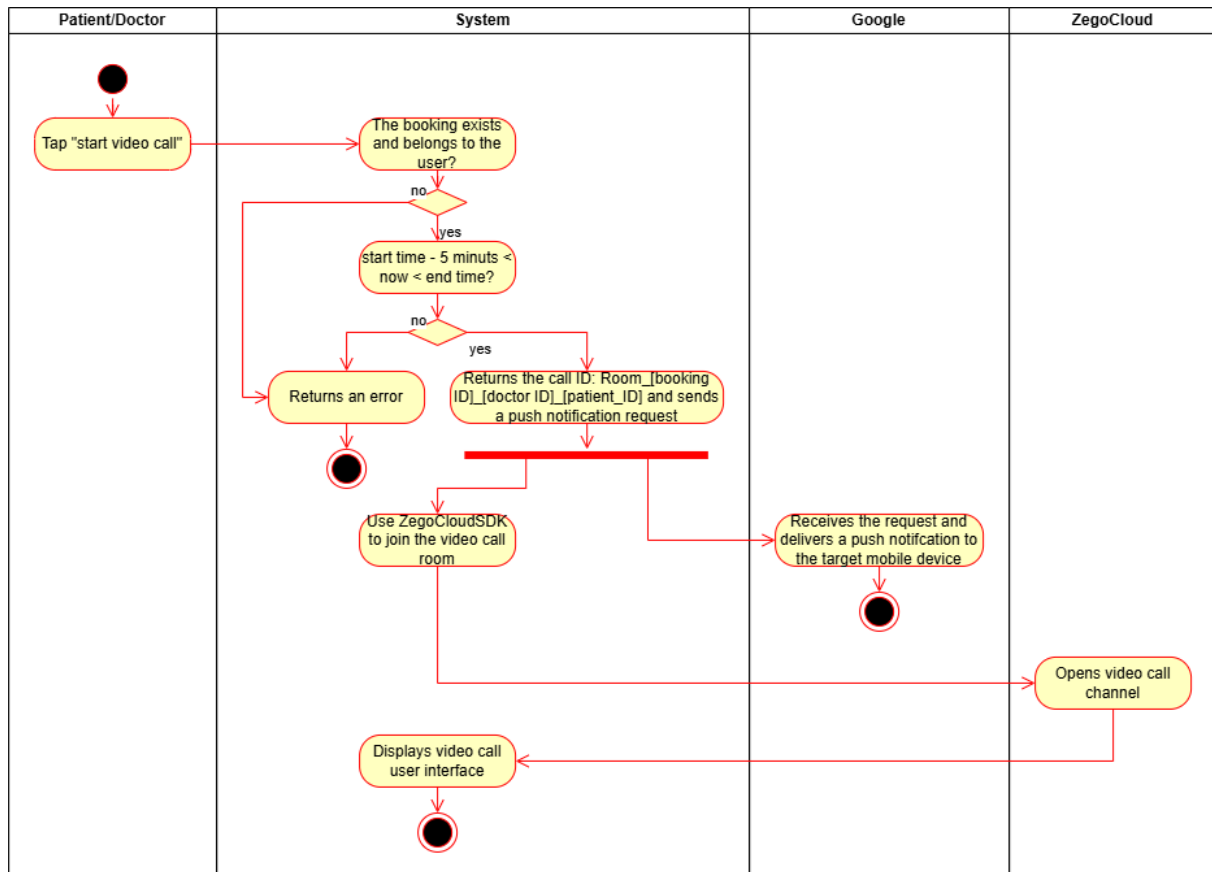


Figure 2. 15. Join remote consultation process

3.6. Create schedule change request

Purpose: Allows doctors to request for permanent adjustments or roster swaps in their recurring weekly schedule template, routing the request to the administrative inbox.

User interface:

Figure 2. 16. Create schedule change request user interface

User interface components:

No.	Control type	Content	Description
1	Button	Back	Return to the previous screen
2	Button	Schedule change request lists	Change to view schedule change request lists
3	Button	Create schedule change request	Change to create schedule change request
4	Button	Add shift	Add shift
5	Button	Delete shift	Delete shift
6	Dropdown	Day of week	Select day of week
7	Dropdown	Clinic type	Select clinic type
8	Time picker	Start time	Select start time
9	Time picker	End time	Select end time

10	Button	Submit	Submit request
-----------	--------	--------	----------------

Table 2. 30. Create schedule change request user interface components

Used data:

No.	Table name	Operation			
		Create	Update	Delete	Read
1	schedule_change_requests	X			X
2	opening_times				X
3	schedule_change_request_details	X			

Table 2. 31. Create schedule change request used data

Process:

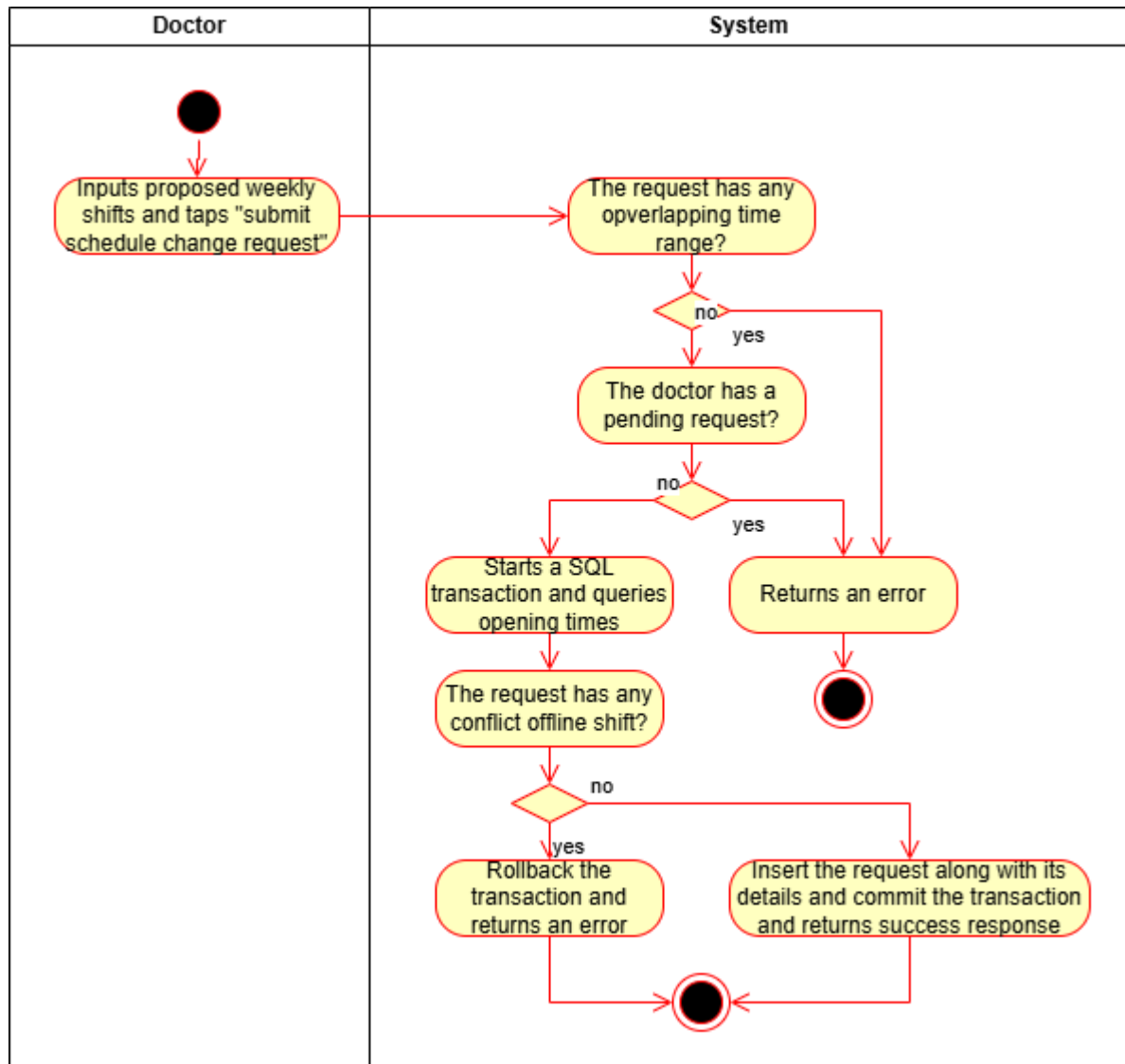


Figure 2. 17. Create schedule change request process

3.7. Handle schedule change request

Purpose: Enables the admin to approve or reject pending doctor roster modifications, automatically updating the doctor's weekly shift template upon approval.

User interface:



Figure 2. 18. Handle schedule change request user interface

User interface components:

No.	Control type	Content	Description
1	Button	View detail	View request details
2	Button	Approve	Approve the request
3	Button	Reject	Reject the request

Table 2. 32. Handle schedule change request user interface components

Used data:

No.	Table name	Operation			
		Create	Update	Delete	Read
1	schedule_change_requests		X		X
2	doctor_weekly_templates	X		X	
3	schedule_change_request_details				X

Table 2. 33. Handle schedule change request used data

Process:

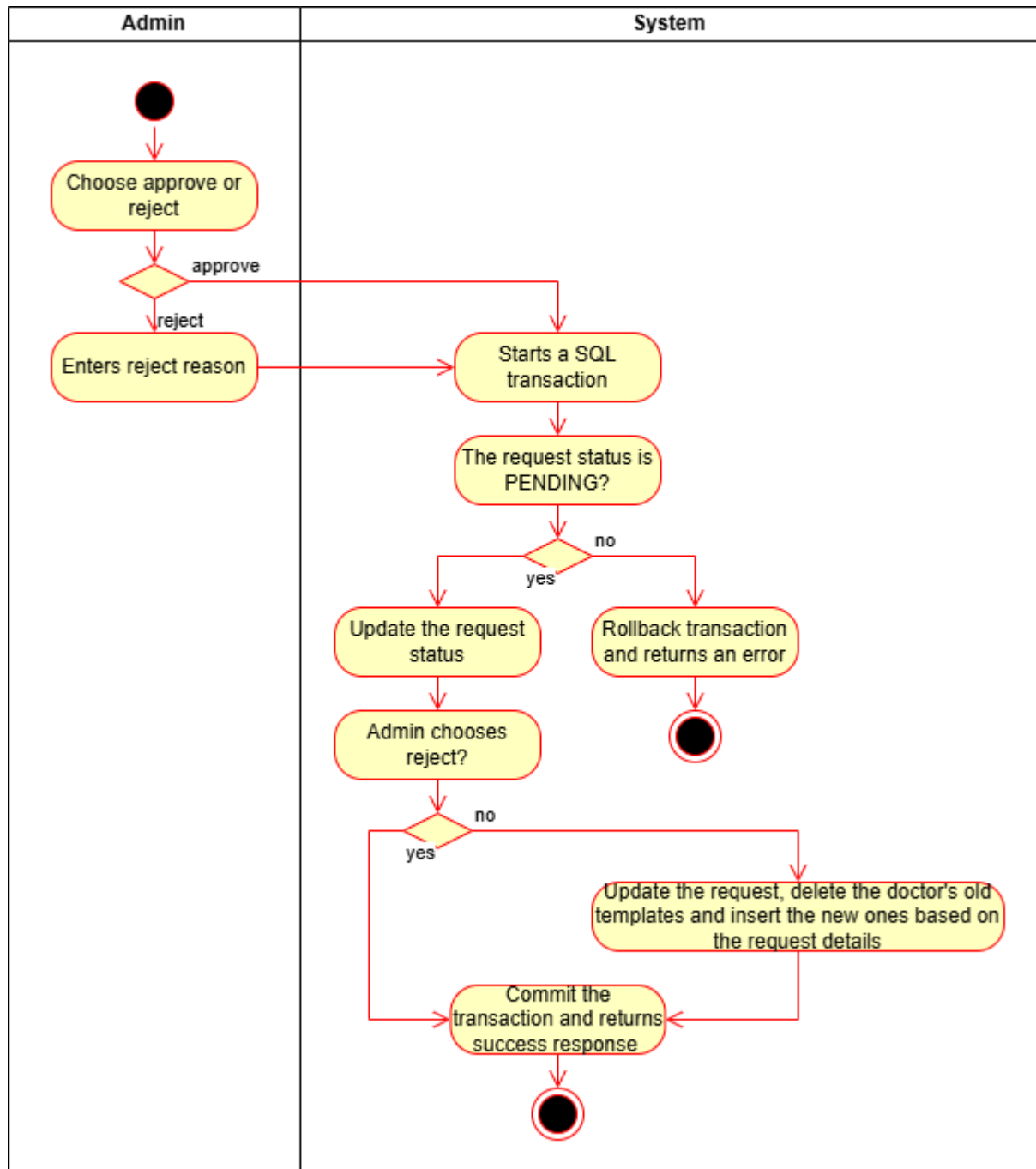


Figure 2. 19. Handle schedule change request process

3.8. Pay invoices online

Purpose: Enables patients to securely settle consultation and medication fees cashless online through integration with the third-party VNPay electronic payment gateway.

User interface:

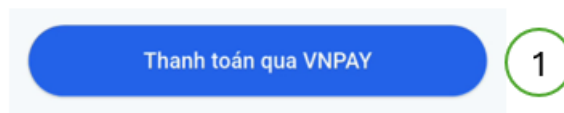


Figure 2. 20. Pay invoices online user interface

User interface components:

No.	Control type	Content	Description
1	Button	Pay with VNPay	Pay with VNPay

Table 2. 34. Pay invoices online user interface components

Used data:

No.	Table name	Operation			
		Create	Update	Delete	Read
1	invoices		X		X
2	payment_transactions	X	X		X

Table 2. 35. Pay invoices online used data

Process:

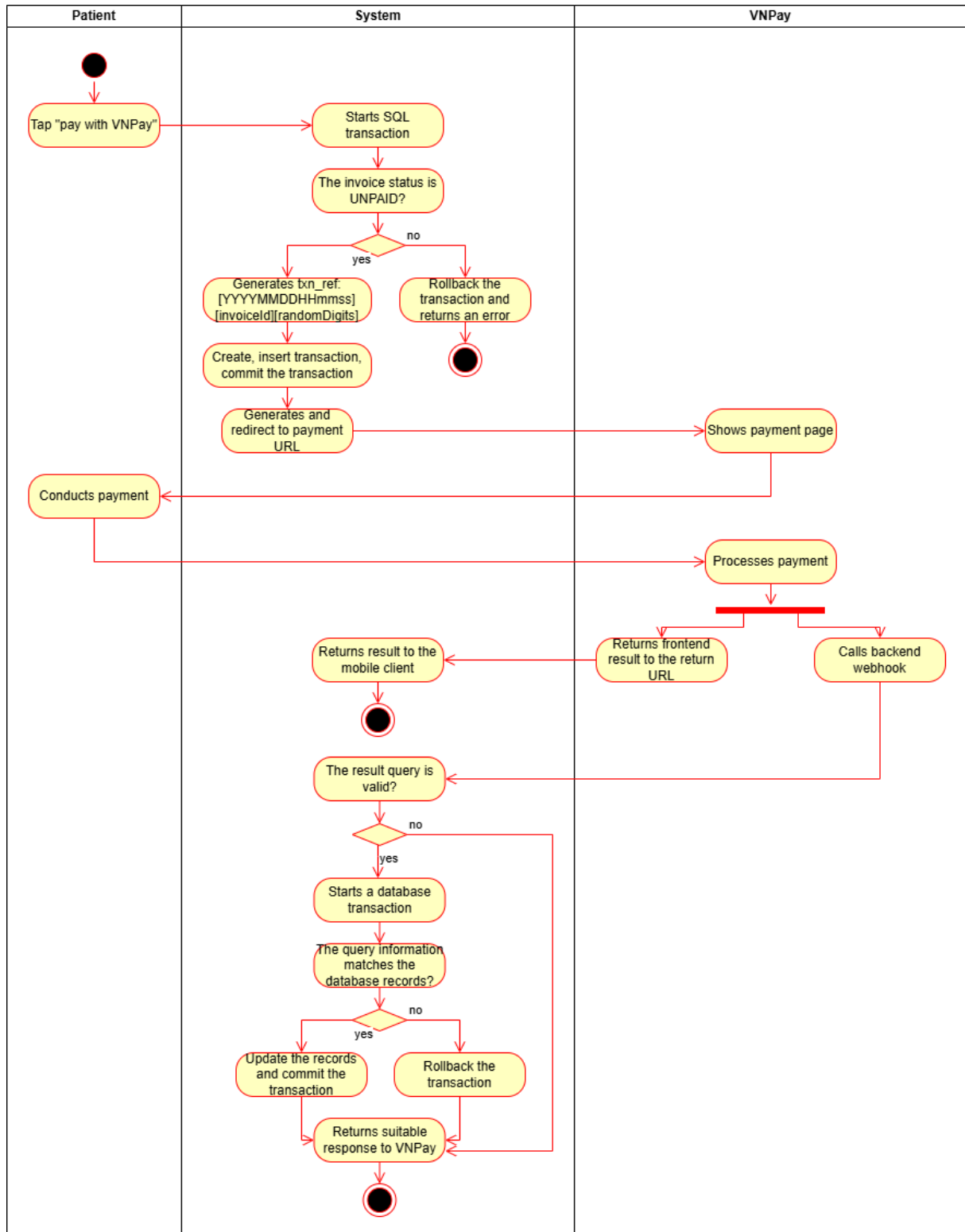


Figure 2. 21. Pay invoices online process

3.9. View invoices

Purpose: To enable authenticated Nurses and Admins to securely retrieve, search, and inspect clinic-wide invoice records and their associated clinical prescription details. For the Nurse, this functionality is essential to verify billing statuses, confirm total amounts, and cross-reference clinical prescriptions during

medicine dispensing and patient checkout workflows. For the Admin, it serves as an administrative audit and oversight tool to monitor financial transactions, track cumulative revenue, and conduct clinical-financial reconciliation.

User interface:

Hóa đơn thanh toán

Quản lý và tra cứu các hóa đơn khám bệnh của bệnh nhân.

Tất cả trạng thái

MÃ HD	MÃ BỆNH NHÂN	PHÍ KHÁM	PHÍ THUỐC	TỔNG TIỀN	TRẠNG THÁI	NGÀY TẠO	
#4	2 Lịch hẹn #12	120.000 đ	0 đ	120.000 đ	Đã trả	05/08/2026	Chi tiết
#1	2 Lịch hẹn #2	150.000 đ	89.000 đ	239.000 đ	Đã trả	15/06/2026	Chi tiết
#2	2 Lịch hẹn #3	120.000 đ	55.000 đ	175.000 đ	Chưa trả	15/06/2026	Chi tiết

Hiện thị 1-3 / 3 bản ghi

< 1 >

Figure 2. 22. View invoices user interface 1

Chi tiết hóa đơn #2

N

Nguyễn Văn A

Chưa thanh toán

Lịch hẹn #3 • Tạo lúc: 15/06/2026 15:14

Chi tiết chi phí

Phí khám bệnh

120.000 đ

Phí thuốc

55.000 đ

Tổng thanh toán

175.000 đ

Xem chi tiết đơn thuốc

Xác nhận thanh toán tiền mặt

Đóng

Figure 2. 23. View invoices user interface 2

User interface components:

No.	Control type	Content	Description
1	Dropdown	Status	Status filter
2	Button	Details	View invoice details
3	Button	Page number	Change page
4	Button	Close	Close modal
5	Button	View prescription	View prescription details
6	Button	Confirm cash payment	Confirm cash payment, only visible to nurses
7	Button	Close	Close modal

*Table 2. 36. View invoices user interface components***Used data:**

No.	Table name	Operation			
		Create	Update	Delete	Read
1	invoices				X
2	users				X
3	medical_records				X
4	diseases				X
5	medicines				X
6	prescription_details				X

Table 2. 37. View invoices used data

Process:

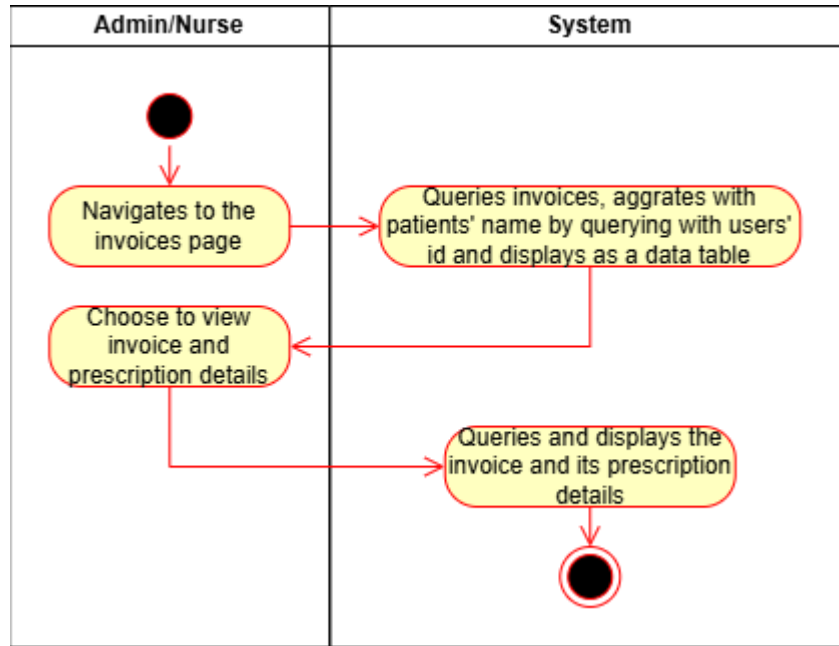


Figure 2. 24. View invoices process

3.10. Confirm cash payment

Purpose: Enables the Nurse to manually verify cash payments at the clinical reception counter, logging their cashier ID and immediately completing the patient checkout cycle.

User interface:

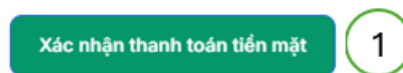


Figure 2. 25. Confirm cash payment user interface

User interface components:

No.	Control type	Content	Description
1	Button	Confirm cash payment	Confirm cash payment

Table 2. 38. Confirm cash payment user interface components

Used data:

No.	Table name	Operation			
		Create	Update	Delete	Read
1	invoices		X		X

Table 2. 39. Confirm cash payment used data

Process:

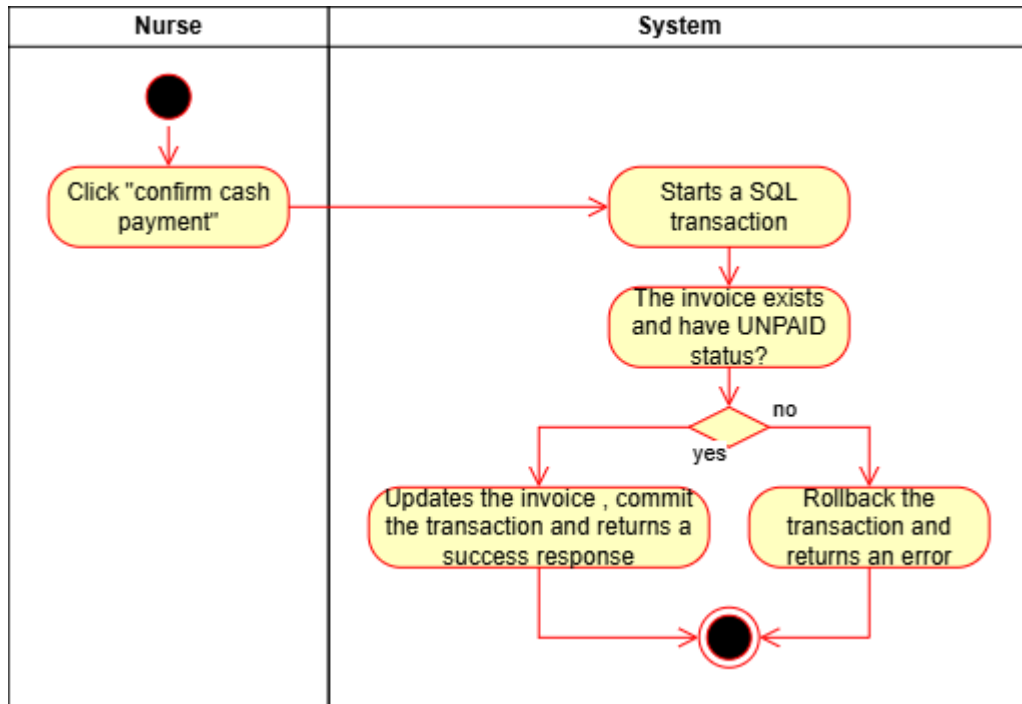


Figure 2. 26. Confirm cash payment process

CHAPTER 3: SOFTWARE TESTING AND PERFORMANCE EVALUATION

1. Introduction

1.1. Testing objectives:

- Functional soundness: To confirm that each of the ten core workflows executes in strict compliance with its defined clinical and administrative business rules, preconditions, and database requirements
- Transactional and scheduling integrity: To verify that the database-level locks during patient booking operations and Redis-backed distributed locks during automated schedule generation prevent write conflicts and overbooking.
- Asynchronous event orchestration: To ensure that background system events – such as RabbitMQ messaging for logging, Firebase notifications, and automated timer cleanup operations – are executed reliably without blocking API response times
- Integration and protocol validation: To validate the security of the API Gateway, the payload efficiency of synchronous internal gRPC communications, the accuracy of VNPay cashless transactions, and the low-latency real-time video connection established via ZegoCloud
- Security and access boundaries: To prove that stateless JWT token validation and RBAC rules are strictly enforced, blocking any unauthorized or cross-role API calls

1.2. Testing scope

During the installation and execution of this program, the functional requirements were verified, and all required functions operated at an acceptable level. However, this report focuses solely on the testing of the system's core functions.

2. Test plan details

2.1. Tested functions

- Book appointment
- Schedule
- Clear unused schedule
- Examine
- Join remote consultation
- Create schedule change request
- Handle schedule change request
- Pay invoices online
- View invoices
- Confirm cash payment

2.2. Approach methods

- Testing pre-requisites and preparation:
 - Rationale: To ensure that the software testing process achieves high efficiency and reliability, several upfront preparation requirements must be established, such as detailed functional use case specifications, professional testing and simulation tools, comprehensive clinical mock datasets, and a stable, isolated staging environment. Without these foundational elements, the testing phase may fail to achieve the required precision, coverage, and depth to validate a distributed microservices architecture
 - Solution: Ensure that all baseline documentation, including clinical workflow definitions and API schemas, is fully completed. Set up a standardized staging environment (utilizing Docker and WSL to isolate services) and seed comprehensive, self-generated datasets (including doctor weekly templates, active schedules, and mock patient profiles) to realistically simulate clinical operations. Utilize robust, standardized tools (such as Android Studio emulators, browser developer tools, and Postman) to optimize functional verification, execution state checks, and system log tracking
- Sequential testing execution lifecycle:
 - Formulating the test plan
 - Preparing test scenarios
 - Constructing detailed test cases
 - Running test execution and monitoring
 - Evaluating and reporting test results
 - Finalizing test documentation and handoff

2.3. Test pass / fail criteria

- Pass criteria:
 - Individual test case pass: A test case is evaluated as successful (Pass) if the actual software execution matches the expected outcomes, complies with clinical business logic, and satisfies the defined specification guidelines
 - Overall phase pass: The benchmark for the overall testing phase to be considered successful is to achieve a functional pass rate of over 95% of all designed test cases, with failed test cases remaining under 5%. Crucially, these failed test cases must be minor anomalies that do not compromise database transactional integrity, role-based access control, or clinical data accuracy, and can be resolved within the remaining testing timeline

- Fail criteria: A test case is evaluated as failed (Fail) when the actual execution output deviates from the expected results, triggers unhandled database or runtime exceptions, or fails to meet the specified business requirements
- Final acceptance criterion: The clinic management and telehealth platform is officially verified as ready for deployment when 100% of the designed test cases achieve a successful "Pass" status after iterative bug-fixing and regression cycles

2.4. Suspension and re-evaluation criteria

- Suspension criteria: The testing process will be suspended immediately if at least one critical, high-impact test case fails. Such critical failures include security authentication breaches (JWT whitelist failure), database lock collisions during appointment booking, or total communication failures in gRPC / RabbitMQ or VNPay / ZegoCloud pipelines. Testing is paused until the developer hotfixes the core issue
- Re-evaluation criteria: Re-evaluation and regression testing are mandatorily executed after the developer resolves the blocking software defect and redeploys the stabilized microservice containers

2.5. Test deliverables

- Strategic test plan
- Detailed test cases
- Defect and evaluation reports

2.6. Testing environment

- Hardware:
 - o Mobile client testing: Samsung Galaxy S25 physical smartphone running Android 16, equipped with Qualcomm Snapdragon 8 Elite for Galaxy, 12 GB RAM, and a 6.2-inch screen to monitor native Flutter performance and RTI camera frames
 - o Web portal and backend staging host: MSI Katana 15 B13VFK – 676VN laptop running Windows 11 (WSL2 with Docker Desktop), equipped with Intel Core i7-13620H, 16 GB RAM, 1 TB SSD, and NVIDIA RTX 4060 to host the NestJS services, MySQL, Redis and RabbitMQ
- Software:
 - o Databases and middleware: MySQL relational databases (decoupled database-per-service), Redis cache (for whitelist caching, cache partitions, and distributed locks), and RabbitMQ event broker for asynchronous message delivery
 - o Development and emulation: Node.js runtime, Android Studio virtual device emulators, Antigravity, and Microsoft Edge web browser (v151)

- Network configuration: Standardized private Docker network bridge with a local bandwidth threshold of at least 60 Mbps

2.7. Test resources

- Integrated toolsets: Android Studio, Antigravity, Git, Ngrok, and GitHub for source code management and deployment
- API testing engine: Postman for writing and running REST API integration tests
- Infrastructure orchestration: Docker and Docker Compose for managing containerized staging components

2.8. Testing risks and mitigation plans

Risk	Impact level	Mitigation strategy
Testing schedule delay	Medium	Increase testing frequency, focus strictly on the ten core transactions, and utilize Postman to accelerate API checking
Staging environment crash	High	Implement containerized backup strategies for MySQL database seeds, keeping Docker-compose and environment configuration files version-controlled in GitHub
Third-party API integration failures	High	Establish mock service boundaries and configure offline simulation responses to verify client flows when external VNPay sandbox or ZegoCloud servers are down
Hardware failure of test devices	High	Prepare secondary backup physical smartphones (running Android 13+) and configure high-performance local emulator images inside Android Studio

Table 3. 1. Testing risks

3. Test scenarios

Project name		Developing An Online Appointment Management, Telehealth Consulting, And Healthcare Support System	
Tester		Duc Hung Nguyen	
Test date		06/08/2026	
Use case ID	Test scenario ID	Description	Number of test cases
UC-01	TS_01	Successful booking	1
	TS_02	Prevent Race Condition (Double	1

		booking)	
	TS_03	Invalid booking	1
UC-02	TS_04	Successful schedule generation	1
	TS_05	Exclude leave days	1
	TS_06	Distributed Lock safety	1
UC-03	TS_07	Delete expired empty slots	1
	TS_08	Skip booked slots	1
UC-04	TS_09	Successful examination completion	1
	TS_10	Rollback on invoice creation failure	1
	TS_11	Examine an already finished booking	1
UC-05	TS_12	Join on time	1
	TS_13	Join too early	1
	TS_14	Join too late	1
UC-06	TS_15	Successful request creation	1
	TS_16	Invalid time frame	1
	TS_17	Existing pending request	1
UC-07	TS_18	Approve request	1
	TS_19	Reject request	1
UC-08	TS_20	Successful payment (IPN returns 00)	1
	TS_21	Prevent duplicate transaction codes	1
	TS_22	Invalid IPN signature	1
UC-09	TS_23	Fetch invoice list	1
	TS_24	View prescription details	1
UC-10	TS_25	Successful cash confirmation	1
	TS_26	Confirm already paid invoice	1

Table 3. 2. Test scenarios

4. Test cases

Test case ID	Test scenario ID	Test content	Steps	Expected result	Obtained result	Evaluation
TC_01	TS_01	Patient books a valid slot	1. Select time slot. 2. Submit booking request.	Booking is successful. A new record is created in the bookings table. time_slots. status changes to BOOKED. API latency < 1.5 s.	Booking is successful. A new record is created in the bookings table. time_slots. status changes to BOOKED. API latency: 75 ms.	PASS
TC_02	TS_02	Two patients attempt to book the same slot simultaneously	1. Patient A and B submit a booking request for the same timeSlotId concurrently.	Due to pessimistic_write lock, only one succeeds. The other receives an error. API latency < 1.5 s.	Due to pessimistic_write lock, only one succeeds. The other receives an error. API latency: 20 ms.	PASS
TC_03	TS_03	Attempt to book an already booked slot	1. Patient submits a booking request for this timeSlotId.	The system returns an error. API latency < 1.5 s.	The system returns an error. API latency: 19 ms.	PASS
TC_04	TS_04	Generate slots for a doctor with a valid template	1. Cronjob executes (or API triggered).	The system generates all TimeSlots (AVAILABLE) for the next X days, matching the configured time intervals.	The system generates all TimeSlots (AVAILABLE) for the next X days, matching the configured time intervals.	PASS
TC_05	TS_05	Doctor has an approved leave	1. Cronjob executes to	Slots are generated normally for other days. No slots are	Slots are generated normally for other days. No slots are	PASS

		request	generate slots including the approved day.	generated for this doctor on the approved day.	generated for this doctor on the approved day.	
TC_06	TS_06	Multiple cronjob nodes run simultaneously	1. Simulate 2 nodes triggering the Cronjob at the exact same second.	Thanks to the Redis Lock (Lock:Scheduling), only one node executes the generation; the other skips. No duplicate slots are generated.	Thanks to the Redis Lock (Lock:Scheduling), only one node executes the generation; the other skips. No duplicate slots are generated.	PASS
TC_07	TS_07	AVAILABLE slots from yesterday	1. Cleanup cronjob executes.	These slots are successfully deleted (or status updated) from the database.	These slots are successfully deleted (or status updated) from the database.	PASS
TC_08	TS_08	BOOKED slots from yesterday	1. Cleanup cronjob executes.	These BOOKED slots are kept (not deleted) to maintain data integrity for medical records.	These BOOKED slots are kept (not deleted) to maintain data integrity for medical records.	PASS
TC_09	TS_09	Valid diagnosis and prescription entered	1. Doctor fills in the medical record and prescription. 2. Clicks Complete.	Medical record is created. Prescription is created (price_at_booking is locked). Invoice is created. Booking status changes to FINISHED. API latency < 1.5 s.	Medical record is created. Prescription is created (price_at_booking is locked). Invoice is created. Booking status changes to FINISHED. API latency: 259 ms.	PASS
TC_10	TS_10	Valid medical data, but Payment Service fails	1. Doctor clicks Complete.	Saga flow activates: Medical record created successfully, invoice creation fails, rollback (Deletes the newly created medical record). Booking remains CONFIRMED. An error is returned to the doctor.	Saga flow activates: Medical record created successfully, invoice creation fails, rollback (Deletes the newly created medical record). Booking remains CONFIRMED. An error is returned to the doctor.	PASS

				API latency < 1.5 s.	API latency: 133 ms.	
TC_11	TS_11	Attempt to call complete API again	1. Submit complete examination request.	System returns an error. API latency < 1.5 s.	System returns an error. API latency: 15 ms.	PASS
TC_12	TS_12	Click join within the valid time window	1. Click "Join Video Call".	Open video call. The system sends a Push Notification (FCM) to the peer's device. API latency < 1.5 s.	Open video call. The system sends a Push Notification (FCM) to the peer's device. API latency: 22 ms.	PASS
TC_13	TS_13	Click join long before the start time	1. Click "Join Video Call".	Return an error. API latency < 1.5 s.	Return an error. API latency: 9 ms.	PASS
TC_14	TS_14	Click join after the slot has ended	1. Click "Join Video Call".	Return an error. API latency < 1.5 s.	Return an error. API latency: 29 ms.	PASS
TC_15	TS_15	OFFLINE slots are within clinic operating hours	1. Doctor selects 08:00 – 12:00 slot. 2. Submits request.	Created successfully. Request status is PENDING. API latency < 1.5 s.	Created successfully. Request status is PENDING. API latency: 245 ms.	PASS
TC_16	TS_16	Submit OFFLINE slot outside clinic hours	1. Doctor selects OFFLINE slot 18:00 – 20:00. 2. Submits request.	Return an error. API latency < 1.5 s.	Return an error. API latency: 54 ms.	PASS
TC_17	TS_17	Doctor tries to create a	1. Submit a new	Return an error. API latency <	Return an error. API latency:	PASS

		request while one is already PENDING	request.	1.5 s.	38 ms.	
TC_18	TS_18	Admin clicks Approve	1. Admin clicks Approve.	Request status changes to APPROVED. The doctor's old templates are deleted, and new templates are inserted into the doctor_weekly_templates table. API latency < 1.5 s.	Request status changes to APPROVED. The doctor's old templates are deleted, and new templates are inserted into the doctor_weekly_templates table. API latency: 66 ms.	PASS
TC_19	TS_19	Admin clicks Reject with a reason	1. Admin Reject. 2. Admin enters reason.	Request status changes to REJECTED with the reason. The doctor's current templates remain unchanged. API latency < 1.5 s.	Request status changes to REJECTED with the reason. The doctor's current templates remain unchanged. API latency: 44 ms.	PASS
TC_20	TS_20	Patient completes payment on VNPay successfully	1. Patient clicks "Pay". 2. IPN webhook returns code 00.	payment_transactions status = SUCCESS. Invoice changes to PAID, payment_method = VNPay.	payment_transactions status = SUCCESS. Invoice changes to PAID, payment_method = VNPay.	PASS
TC_21	TS_21	Patient clicks pay multiple times creating multiple links	1. Patient clicks "Pay" 3 times.	Backend generates 3 completely different txnRef codes. No duplicate reference error from VNPay.	Backend generates 3 completely different txnRef codes. No duplicate reference error from VNPay.	PASS
TC_22	TS_22	Malicious actor fakes an	1. Send IPN Callback API	Backend detects invalid HMAC-SHA512 signature.	Backend detects invalid HMAC-SHA512 signature.	PASS

		IPN callback	with a fake vnp_Secure Hash.	Returns error code 97. Invoice status remains unchanged.	Returns error code 97. Invoice status remains unchanged.	
TC_23	TS_23	Admin accesses the invoices page	1. Go to invoices page.	System returns the invoice list, including patient names (full_name) populated via gRPC from the User Service. API latency < 3 s.	System returns the invoice list, including patient names (full_name) populated via gRPC from the User Service. API latency: 44 ms.	PASS
TC_24	TS_24	Admin clicks "View Details" on an invoice	1. Click "View Details". 2. Click "View Prescription Details".	API Gateway calls Medical Record Service. Returns full prescription details. API latency < 3 s.	API Gateway calls Medical Record Service. Returns full prescription details. API latency: 66 ms.	PASS
TC_25	TS_25	Nurse confirms payment for an unpaid invoice	1. Nurse clicks "Confirm Cash Paid".	Invoice changes to PAID, payment_method = CASH, and records cashier_id with the Nurse's ID. API latency < 1.5 s.	Invoice changes to PAID, payment_method = CASH, and records cashier_id with the Nurse's ID. API latency: 25 ms.	PASS
TC_26	TS_26	Nurse attempts to confirm an invoice already paid via VNPay	1. Nurse (accidentally/due to lag) clicks "Confirm Cash Paid".	System uses pessimistic_write lock and checks status. Returns an error. Payment method is not overwritten. API latency < 1.5 s.	System uses pessimistic_write lock and checks status. Returns an error. Payment method is not overwritten. API latency: 18 ms.	PASS

Table 3. 3. Test cases

5. Test result evaluation

The functions within the scope of testing cover all test cases. The overall relationship accurately reflects the required operation of the system.

Test scenario ID	Description	Number of test cases	Number of successful test cases	Number of failed test cases
TS_01	Successful booking	1	1	0
TS_02	Prevent Race Condition (Double booking)	1	1	0
TS_03	Invalid booking	1	1	0
TS_04	Successful schedule generation	1	1	0
TS_05	Exclude leave days	1	1	0
TS_06	Distributed Lock safety	1	1	0
TS_07	Delete expired empty slots	1	1	0
TS_08	Skip booked slots	1	1	0
TS_09	Successful examination completion	1	1	0
TS_10	Rollback on invoice creation failure	1	1	0
TS_11	Examine an already finished booking	1	1	0
TS_12	Join on time	1	1	0
TS_13	Join too early	1	1	0
TS_14	Join too late	1	1	0
TS_15	Successful request creation	1	1	0
TS_16	Invalid time frame	1	1	0
TS_17	Existing pending request	1	1	0
TS_18	Approve request	1	1	0

TS_19	Reject request	1	1	0
TS_20	Successful payment (IPN returns 00)	1	1	0
TS_21	Prevent duplicate transaction codes	1	1	0
TS_22	Invalid IPN signature	1	1	0
TS_23	Fetch invoice list	1	1	0
TS_24	View prescription details	1	1	0
TS_25	Successful cash confirmation	1	1	0
TS_26	Confirm already paid invoice	1	1	0

Table 3. 4. Test results

PART III. CONCLUSION

1. Achieved results

1.1. Personal growth and technical development

Throughout the research, architectural design, and implementation phases of developing this project, I have successfully bridged the gap between theoretical software engineering principles and complex modern implementations. This project has served as an invaluable milestone for my professional development, particularly as it challenged me to quickly familiarize myself with, master, and integrate several advanced frameworks and tools for the very first time.

Transitioning to TypeScript allowed me to appreciate the significance of a strongly typed language in mitigating runtime errors and maintaining rigorous code readability in enterprise applications. Building my first server-side codebase using NestJS helped me master progressive architectural patterns, including dependency injection, modular folder structures, and native microservice decorators, providing a highly scalable and clean foundation for the backend tier.

Overcoming the challenges of traditional, fragile local databases, I learned to containerize and manage core middleware infrastructure using Docker. Setting up isolated, reproducible container images for MySQL databases, a Redis cache, and a RabbitMQ message broker on WSL guaranteed a stable local staging environment.

Implementing remote peer-to-peer real-time communication was a completely new technical domain for me. By researching and integrating ZegoCloud's SDK, I successfully handled complex signaling pipelines, media stream rendering, and network transit negotiations to establish secure, low-latency video rooms on Flutter.

To support clinical responsiveness, I gained hands-on experience configuring FCM. I successfully connected FCM with my backend event queues inside RabbitMQ, enabling the system to trigger secure, instantaneous mobile alerts to patient and doctor devices without blocking the primary application threads.

Beyond individual tools, I developed a strong architectural mindset. I mastered internal communication protocols, using gRPC over HTTP/2 with protocol buffers to minimize payload size during internal lookups. Furthermore, I designed robust concurrency strategies, implementing MySQL row-level locking to protect clinical slots during simultaneous booking attempts and utilizing Redis distributed locks to orchestrate automated scheduler daemons safely.

1.2. Program practically and real-world application

The developed platform has been realized as a fully functional, containerized, and highly stable clinic management and telehealth prototype, ready for localized deployment in private outpatient facilities.

The system replaces obsolete, manual, paper-based administrative queuing workflows with an automated, self-service booking system on the Flutter mobile client. This systematically eliminates clinic overcrowding, long waiting room delays, and administrative booking conflicts.

By incorporating ZegoCloud RTI video consultations, the system breaks down geographical barriers and expands healthcare accessibility beyond physical clinic hours, providing a seamless remote care pipeline for non-emergency or follow-up consultations.

The backend securely coordinates clinical and financial domains via gRPC, automatically generating unpaid invoices based on doctor diagnostic entry and pharmaceutical prescription records. This allows nurses to instantly retrieve prepared orders at the pharmacy counter or patients to immediately complete cashless online checkout via VNPay.

The platform ensures robust medical-financial transaction security during the critical consultation completion phase. By utilizing the schedule service to orchestrate transactions across the medical record service and the payment service, the system guarantees that no partial or orphaned records are ever committed. If invoice generation fails, the Saga compensations instantly revert the booking and time slot status, ensuring perfect distributed database synchronization.

Rather than hosting raw, unisolated database engines on the host operating system, the platform ensures execution safety and data integrity by isolating the database-per-service layer, the message broker exchange, and the memory caching layer inside dedicated Docker containers. This containerized backend infrastructure guarantees high performance and clean environmental recovery during local system staging.

2. Limitations and difficulties

Despite successfully meeting the primary functional specifications and validating the core clinical workflows, the system exhibits certain technical limitations and operational challenges under practical execution.

During the client-side engineering phase, compiling and launching the Flutter mobile application for local debugging proved to be exceptionally time-consuming. Cold starts, Gradle resource resolution, and dependency linking on multiple endpoints imposed a severe tax on development velocity. The long compilation cycles hindered rapid UI/UX adjustments.

Simulating real-time clinic notifications using Android Studio virtual emulators frequently introduced frustrating development anomalies. Due to shared local network socket footprints, virtual device identifier overlaps, or caching errors within the emulator's sandbox environment, FCM occasionally experienced cross-role notification delivery errors. Under certain concurrent testing scenarios, notifications intended for a patient emulator were incorrectly routed to a doctor emulator (and vice-versa), presenting a major debugging obstacle that could not be solved within the virtualized environment alone.

The ZegoCloud SDK utilized for remote video consultations was fundamentally incompatible with virtualized Android Studio emulators. Emulators consistently failed to map virtualize physical camera / microphone hardware feeds, resulting in end-to-end verification of the telehealth consults could only be realized by deploying the application onto physical mobile hardware, where the real-time video rooms operated flawlessly with low latency.

During the client-side development of the mobile application for doctors and patients, configuring the Flutter environment introduced significant package build blockages. Unlike clean, default Flutter projects, integrating multiple highly specialized native plugins – such as the ZegoCloud SDK for video consultations and FCM for push alerts for token persistence – resulted in severe version conflicts within the Gradle build tier. Resolving these compile-time anomalies required manually overriding default Kotlin versions, modifying Android-specific properties, which deviated heavily from standard project templates.

Implementing external pipelines revealed major inconsistencies within the technical documentation provided by third-party service providers. Their official guides were frequently found to be outdated, ambiguous, or misaligned with recent library releases, requiring extensive trial-and-error to align with the NestJS gateway microservice tier.

The reliability and quality of the telehealth consultation module are heavily dependent on the ZegoCloud signaling network and the user's physical network conditions. Under unstable network environments, video streams may experience frame drops or latency. Furthermore, because the cashless billing system is integrated with the VNPay Sandbox environment, transaction responses and asynchronous IPN callbacks are subject to simulated processing speeds and occasional staging server downtime beyond the control of the local system.

Hosting seven distinct backend microservices concurrently under Node.js runtime, while simultaneously running separate dedicated databases and brokers inside Docker containers on a single development machine, demands substantial physical memory allocation. This resource-intensive stack can result in high hardware utilization during local multi-service testing.

Although the entire shared infrastructure tier is fully isolated and run within Docker containers, the individual NestJS microservices are currently executed directly on the native host Node.js environment. This partial containerization presents minor deployment and portability hurdles when migrating the application tier across heterogeneous physical server nodes.

3. Future works

To transition the current prototype of the online appointment management, telehealth consulting, and healthcare support system into an enterprise-grade, highly resilient, and intelligent clinical platform, several strategic research and development directions are proposed:

- **Cloud-native orchestration and Kubernetes deployment:** Currently, only the shared infrastructure tier runs inside Docker containers, while the individual NestJS application services execute within the host's native Node.js runtime. Future iterations will focus on fully containerizing all seven independent NestJS microservices into secure Docker images and deploying them within a Kubernetes cluster. This migration will introduce automated horizontal pod autoscaling to seamlessly manage morning appointment registration spikes, automated rolling updates with zero staging downtime, and robust self-healing container recoveries
- **Clinical decision support via AI integration:** To fully realize the "healthcare support" capabilities of the platform, a dedicated AI microservice will be developed. By leveraging natural language processing or fine-tuned clinical large language models, this service will analyze patient-reported symptoms during booking to automatically suggest preliminary disease and relevant historical medical files to the examining physician, thereby optimizing diagnostic accuracy and streamlining clinical workflows
- **Multi-platform expansion and web portal patients:** To eliminate download barriers for patients, a fully responsive patient-facing web application will be developed using ReactJS to mirror the core functionalities of the Flutter mobile client. Concurrently, the mobile codebase will undergo compiling optimizations and native profile integrations to support official iOS deployment, ensuring a broader and more inclusive user demographic
- **Hybrid online-offline queue synchronization:** To bridge the gap between digital healthcare and traditional clinic operations, the platform will implement a unified hybrid queuing engine. This subsystem will seamlessly synchronize online bookings with offline, physical walk-in ticketing devices. The API gateway will dynamically merge both traffic streams into a single, real-time clinical waiting queue ledger, ensuring fair waiting times and preventing scheduling overlaps at the nurse reception desk
- **Low-friction guest booking pipeline:** To accommodate elderly, non-tech-savvy, or first-time patients who face friction with digital registration, the

Flutter mobile client and a future web portal will support a secure guest booking flow. Patients will be permitted to reserve a medical slot without undergoing an upfront, mandatory account registration process. Authentication and identity verification will be handled via a fast, automated OTP transaction linked to their phone number, drastically reducing registration abandonment rates

- **Active rescheduling and dynamic temporal flexing:** The current scheduler strictly binds clinical consultations to rigid, fixed-duration slots. Future iterations will introduce a dynamic rescheduling engine, empowering both doctors and patients with greater temporal flexibility. Users will be permitted to actively postpone, exchange, or fast-track scheduled appointments in real time based on active clinic throughput, dynamically updating the status and time parameters of active bookings without disrupting the master clinic roster
- **Comprehensive pharmaceutical inventory and warehouse management:** The system's database currently operates a static drug registry strictly for prescription writing. Future developments will expand this into a full-scale pharmaceutical inventory management system. This module will track physical stock-level thresholds across clinic warehouses, record drug batches and expiry dates, and initiate automated gRPC commands to deduct item quantities in real time when a doctor submits a prescription during examinations
- **UI/UX customization and internationalization:** To improve usability, reduce eye strain during night shifts for clinical staff, and cater to diverse patient demographics, both Flutter and ReactJS interfaces will be upgraded. This includes a system-wide, dynamic dark mode toggle and localized internationalization, supporting Vietnamese, English, and other regional languages to provide a more accessible, inclusive user experience

PART IV. REFERENCES

- [1] Alfred V. Aho, John E. Hopcroft, Jeffrey D. Ullman, *Data Structures and Algorithms*. Addison-Wesley, 1983.
- [2] Docker, “Docker Documentation.” Accessed: Jun. 16, 2026. [Online]. Available: <https://docs.docker.com>
- [3] GitHub, “GitHub Documentation.” Accessed: Jun. 07, 2026. [Online]. Available: <https://docs.github.com>
- [4] Google, “Firebase Cloud Messaging Documentation.” Accessed: Jul. 18, 2026. [Online]. Available: <https://firebase.google.com/docs/cloud-messaging>
- [5] Google, “Flutter Documentation.” Accessed: Jul. 26, 2026. [Online]. Available: <https://flutter.dev>
- [6] Internet Engineering Task Force (IETF), “IETF RFC 7519: JSON Web Token (JWT),” May 2015. Accessed: Jun. 26, 2026. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>
- [7] Meta, “React Documentation.” Accessed: Jun. 27, 2026. [Online]. Available: <https://react.dev>
- [8] Microsoft, “TypeScript Documentation.” Accessed: Jun. 22, 2026. [Online]. Available: <https://www.typescriptlang.org/docs>
- [9] Microsoft, “Windows Subsystem for Linux (WSL) Documentation.” Accessed: Jun. 22, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/windows/wsl/>
- [10] NestJS, “NestJS Documentation.” Accessed: Jun. 16, 2026. [Online]. Available: <https://docs.nestjs.com>
- [11] Oracle, “MySQL Documentation.” Accessed: Jun. 04, 2026. [Online]. Available: <https://dev.mysql.com/doc>
- [12] Postman, “Postman Documentation.” Accessed: Jul. 27, 2026. [Online]. Available: <https://learning.postman.com/docs/>
- [13] RabbitMQ, “RabbitMQ Documentation.” Accessed: Jun. 04, 2026. [Online]. Available: <https://www.rabbitmq.com/documentation.html>
- [14] Ralph Rowland Young, *The Requirements Engineering Handbook*. Artech House, 2004.
- [15] Redis, “Redis Documentation.” Accessed: Jun. 17, 2026. [Online]. Available: <https://redis.io/docs/>
- [16] Software Freedom Conservancy, “Git Documentation.” Accessed: Jun. 30, 2026. [Online]. Available: <https://git-scm.com/doc>

- [17] steenbakker.dev, “Flutter Secure Storage Documentation.” Accessed: Jul. 12, 2026. [Online]. Available: https://pub.dev/packages/flutter_secure_storage
- [18] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms, 3rd Edition*. MIT Press, 2009.
- [19] VNPay, “VNPay API Integration Documentation.” Accessed: Jun. 01, 2026. [Online]. Available: <https://sandbox.vnpayment.vn/apis/>
- [20] Winston, “Winston Documentation.” Accessed: Jun. 08, 2026. [Online]. Available: <https://github.com/winstonjs/winston>
- [21] ZegoCloud, “ZegoCloud Documentation.” Accessed: Jul. 19, 2026. [Online]. Available: <https://docs.zegocloud.com>